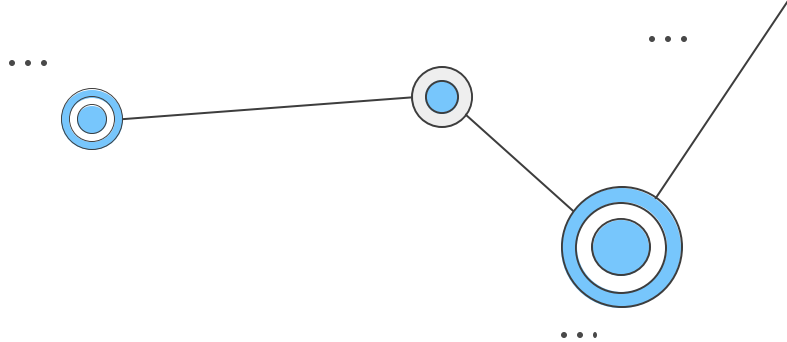
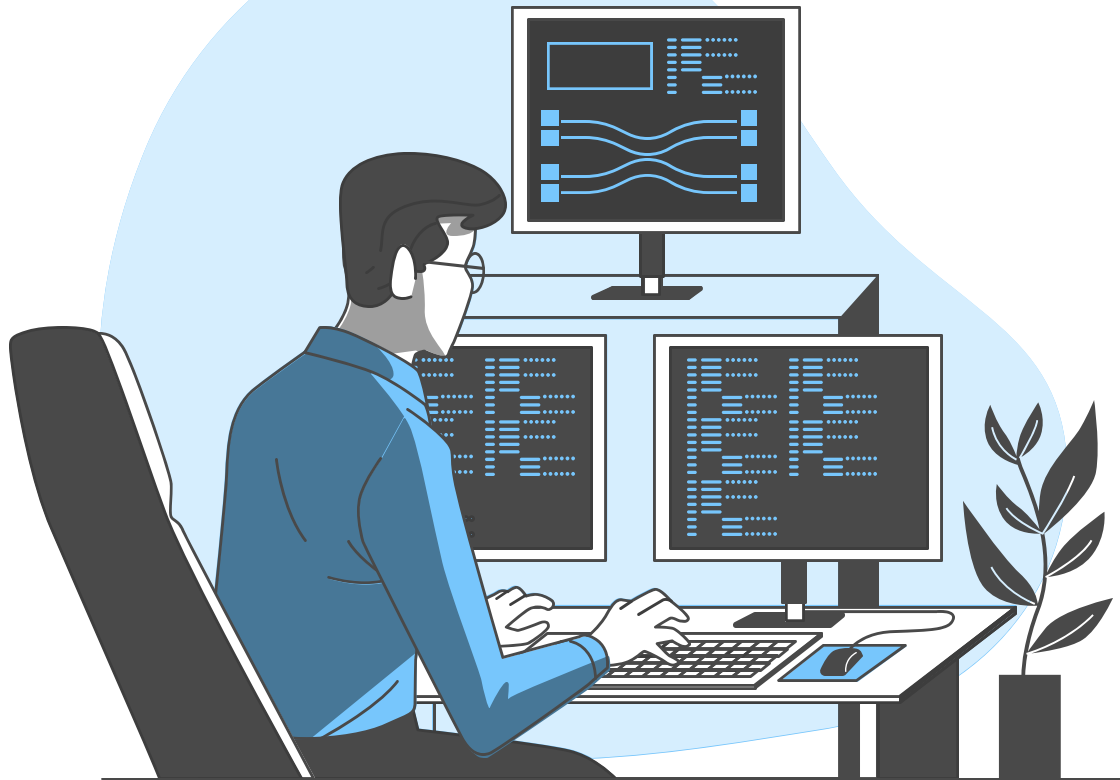




PUC Minas



Laboratório de Desenvolvimento de Software

Prof. Dr. João Paulo Aramuni

Sumário

- Segurança
 - Introdução à Segurança na Web
 - JSON Web Token (JWT)
 - Autenticação e Autorização

Sumário

- Segurança
 - **Introdução à Segurança na Web**
 - JSON Web Token (JWT)
 - Autenticação e Autorização

Segurança

- Introdução à Segurança na Web
 - A segurança em aplicações web é um aspecto **crítico** e indispensável na era digital.
 - Com o crescente volume de informações sensíveis sendo compartilhadas e processadas online, a necessidade de proteger esses dados contra ameaças cibernéticas nunca foi tão crucial.



Segurança

- Introdução à Segurança na Web
 - A **confidencialidade**, **integridade** e **disponibilidade** dos dados são pilares fundamentais para estabelecer a confiança dos usuários e garantir o bom funcionamento de sistemas web. As aplicações web, por sua natureza interconectada e acessível, tornam-se alvos atrativos para cibercriminosos.



Segurança

- Introdução à Segurança na Web
 - Diversos desafios de segurança surgem em ambientes online, incluindo ataques de injeção, como **SQL injection** e **Cross-Site Scripting (XSS)**, roubo de identidade, exposição indevida de dados, vazamentos de dados, entre outros.
 - A implementação de práticas de segurança eficazes não é apenas uma medida reativa para mitigar riscos, mas também uma abordagem proativa para construir aplicações robustas desde o início.

Segurança

- Introdução à Segurança na Web
 - A autenticação e autorização adequadas, o gerenciamento de sessões, a proteção contra vulnerabilidades conhecidas e a comunicação segura são elementos-chave para garantir a segurança em aplicações web.
- De 2020 para cá, os ataques hackers têm aumentado consideravelmente. Em especial, os chamados sequestros de dados com resgate em criptomoedas.

FOLHA DE S. PAULO

★★★

tec > games cibersegurança inteligência artificial smartphones

Oferta Especial: R\$1,90 no 1º mês

ASSINE A FOLHA

BLOOMBERG • CRIMES VIRTUAIS • INTERNET

Clorox, Boeing, MGM e mais: por que grandes ataques hackers aumentaram em 2023

Ocorrências de sequestro de dados de grandes empresas aumentaram em 51% até novembro

f

whatsapp

f

twitter

bookmarks

comment

more

18.dez.2023 às 10h00

OLHAR DIGITAL

NOTÍCIAS VÍDEOS FICHAS TÉCNICAS DICAS E TUTORIAIS EDITORIAIS OFERTAS

Olhar Digital > Segurança e Privacidade > Hackers roubaram informações financeiras de mais de 80 mil pessoas nos EUA

SEGURANÇA E PRIVACIDADE

Hackers roubaram informações financeiras de mais de 80 mil pessoas nos EUA

Os hackers tiveram acesso a números de contas bancárias, de cartões de crédito, códigos de segurança e até senhas

Alessandro Di Lorenzo | 27/12/2023 17h39, atualizada em 02/01/2024 15h28

invest news

CRIPTONews COTAÇÕES

CAFEÍNA

Os ataques hackers que impactaram o mercado financeiro

Sony, Natura e Yahoo! já sofreram com invasões digitais. Conheça os casos, neste Cafeína

Publicado 1 semana atrás em 8 de janeiro de 2024 - Atualizado 1 semana atrás

por Redação InvestNews

CRYPTO ID

ÚLTIMAS NOTÍCIAS

QUEM SOMOS

RECEBA NOVIDADES

CERTIFICADO DIGITAL

TLS

IAM

IA

OT

IDENTIDADE

Brasil em alerta: aumento de ataques de hackers exige maiores investimentos em cibersegurança

20 de outubro de 2023

Pesquisa aponta que o país sofreu 46 bilhões de ataques no 1º semestre de 2023, sendo a segunda nação mais vulnerável a ataques cibernéticos

Menu

O GLOBO | Tecnologia

ASSINE

Buscar

Entrar

Economia / Tecnologia

Brasil é o maior alvo de ataques cibernéticos na América Latina. Veja ranking

País sofreu com alta de 19% no número de ataques no segundo semestre de 2022. Percentual é superior ao observado na média global (13%) e representa quase 40% do total das invasões na América Latina

Por Carolina Nalin — Rio

09/06/2023 18h00 - Atualizado há 7 meses

f

x

whatsapp



Negócios

Google diz que sofreu um dos maiores ataques hackers da história

Segundo o Google, hackers vêm tentando, pelo menos desde agosto, forçar uma parada total nos servidores da empresa. Amazon também foi alvo

Fábio Matos

12/10/2023 16:18, atualizado 12/10/2023 16:22



exame. FUTURE OF MONEY

Assine

Entrar

Future of Money

Home > Future of Money

Ataques hackers serão mais sofisticados em 2024, com técnicas ainda desconhecidas

Após ano de recuperação de preços e fortalecimento da confiança na indústria, devemos ver medidas jurídicas serem implementadas; ação de criminosos deve ficar mais sofisticada

Mundo

Israel-Hamas: hackers abrem trincheiras virtuais, e ciberataques viram arma de guerra

'Combatentes' digitais miram ataques contra sistemas críticos e derrubam sites de organizações humanitárias que recebem doações

Por Juliana Causin — São Paulo

23/10/2023 04h30 · Atualizado há 2 meses



NOTÍCIAS

VÍDEOS

FICHAS TÉCNICAS

DICAS E TUTORIAIS

EDITORIAS

OFERTAS

Olhar Digital > Segurança e Privacidade > Ubisoft investiga possível ataque hacker

SEGURANÇA E PRIVACIDADE

Ubisoft investiga possível ataque hacker

A editora francesa de games está investigando uma possível invasão cibernética após imagens internas da empresa vazarem na internet

Por William Schendes, editado por Gabriel Sêrvio | 25/12/2023 14h00

NOTÍCIAS

VÍDEOS

FICHAS TÉCNICAS

DICAS E TUTORIAIS

EDITORIAS

OFERTAS

Olhar Digital > Segurança e Privacidade > Ataques cibernéticos aumentam em 2023; veja principais erros de cibersegurança

SEGURANÇA E PRIVACIDADE

Ataques cibernéticos aumentam em 2023; veja principais erros de cibersegurança

Especialista aponta que erros básicos de proteção podem levar a ataques cibernéticos e explica como se proteger

Por Vitoria Lopes Gomez, editado por Bruno Capezzi | 19/12/2023 00h20



Economia

Portos australianos continuam parados e contêineres se acumulam após ataque hacker

Interrupção das operações devem continuar por mais alguns dias. Incidente é um lembrete do sério risco que os ataques cibernéticos, escreveu a ministra de Assuntos Internos, Clare O'Neil, na rede social



Por Bloomberg — Sydney

12/11/2023 09h34 · Atualizado há 2 meses



TECNOLOGIA

Governo quer formar 30 mil 'hackers' para hub de cibersegurança no Brasil

Segundo analistas, o salário de um gerente de segurança cibernética em uma grande empresa brasileira hoje não é inferior a 35 mil reais

POR MAURÍCIO THUSWOHL | 16.01.2024 10H22

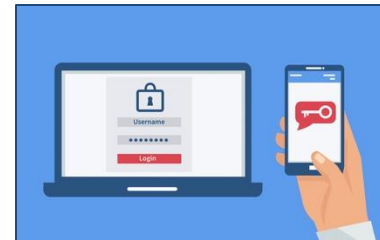


Para não ficar para trás nessa corrida global, o governo brasileiro lançará, em **22 de janeiro**, os cursos do programa **Hackers do Bem**. O objetivo é sanar o déficit de profissionais do setor hoje no País e criar um **hub nacional de cibersegurança**. A iniciativa do **Ministério da Ciência, Tecnologia e Inovação (MCTI)**, em parceria com o **Senai-SP**, contará com recursos do **Programa Prioritário em Informática (PPI)** da Softex e será executada pela organização social **Rede Nacional de Ensino e Pesquisa (RNP)**.



(Foto: iStock)

- Introdução à Segurança na Web
 - Existem diversas práticas recomendadas que contribuem para a segurança de aplicações web. Vejamos algumas delas:
 - **1) HTTPS (SSL/TLS):** Utilizar uma conexão segura por meio do protocolo HTTPS para criptografar a comunicação entre o cliente e o servidor, protegendo contra ataques como Man-in-the-Middle.
 - **2) Autenticação forte:** Implementar métodos robustos de autenticação, como a utilização de senhas fortes, autenticação de dois fatores (2FA) e autenticação biométrica, quando aplicável.
 - **3) Gerenciamento adequado de sessões:** Garantir que as sessões de usuário sejam gerenciadas de forma segura, com práticas como a expiração de sessões inativas, renovação de tokens e o uso de tokens de sessão seguros.



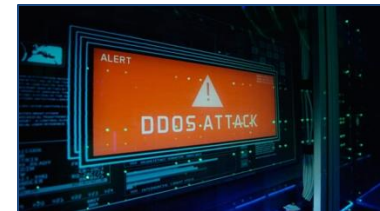
- Introdução à Segurança na Web
 - Existem diversas práticas recomendadas que contribuem para a segurança de aplicações web. Vejamos algumas delas:
 - **4) Proteção contra injeções (SQL, XSS, CSRF):** Validar e sanitizar entradas do usuário para evitar ataques de injeção, como SQL injection. Além disso, proteger contra Cross-Site Scripting (XSS) e Cross-Site Request Forgery (CSRF) com medidas como Content Security Policy (CSP) e tokens anti-CSRF.
 - **5) Controle de acesso e autorização:** Implementar um sistema de controle de acesso granular, garantindo que cada usuário tenha apenas as permissões necessárias. Utilizar **tokens JWT** e definir regras de autorização de forma eficaz.



- Introdução à Segurança na Web
 - Existem diversas práticas recomendadas que contribuem para a segurança de aplicações web. Vejamos algumas delas:
 - **6) Atualizações e patching:** Manter todos os componentes do sistema, incluindo bibliotecas, frameworks e servidores, atualizados com as últimas correções de segurança.
 - **7) Validação de dados no lado do cliente e do servidor:** Realizar validações tanto no lado do cliente quanto no lado do servidor para garantir a integridade dos dados e prevenir manipulações maliciosas.
 - **8) Auditoria e monitoramento:** Implementar logs detalhados e realizar monitoramento contínuo para identificar padrões incomuns, potenciais ameaças e atividades suspeitas.



- Introdução à Segurança na Web
 - Existem diversas práticas recomendadas que contribuem para a segurança de aplicações web. Vejamos algumas delas:
 - **9) Criptografia adequada:** Utilizar criptografia forte para proteger dados sensíveis, como senhas, em repouso e em trânsito. Armazenar senhas de forma segura, utilizando funções de hash com salt.
 - **10) Segurança de arquivos e uploads:** Validar e limitar tipos de arquivos aceitos para upload. Garantir que o sistema de armazenamento de arquivos seja seguro para evitar execução de código malicioso.
 - **11) Firewalls e proteção contra DDoS:** Implementar firewalls para filtrar tráfego indesejado e proteger contra ataques de negação de serviço distribuído (DDoS).

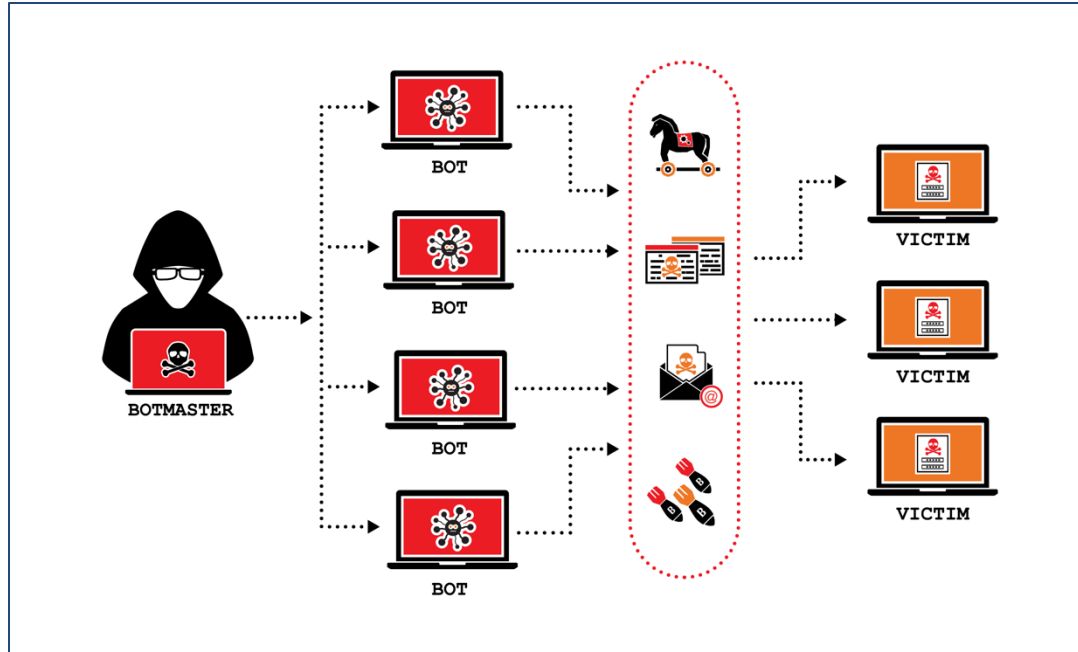


- Introdução à Segurança na Web
 - Existem diversas práticas recomendadas que contribuem para a segurança de aplicações web. Vejamos algumas delas:
 - **12) Treinamento de desenvolvedores e usuários:** Promover a conscientização de segurança por meio de treinamentos regulares para desenvolvedores e usuários, destacando as boas práticas de segurança.
 - **13) CAPTCHAs e Desafios de Reconhecimento Humano:** Introduzir desafios de CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) em áreas críticas da aplicação, como formulários de login e registros.



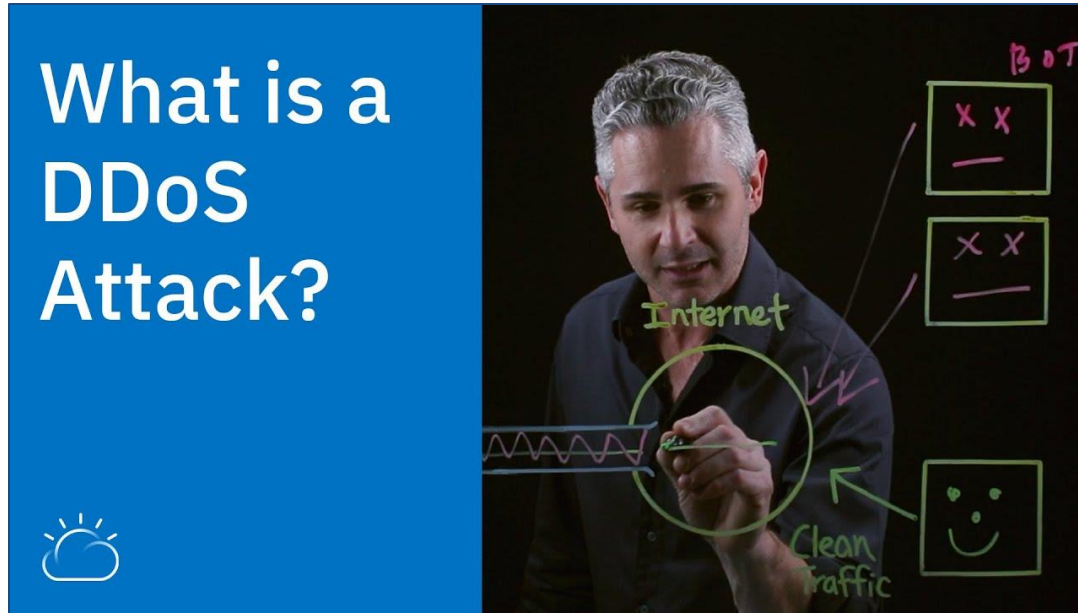
Segurança

- Leitura sugerida:
 - <https://www.thesslstore.com/blog/what-is-a-ddos-attack/>



Segurança

- Vídeo sugerido:
 - https://www.youtube.com/watch?v=z503nLsfe5s&ab_channel=IBMTechology



Segurança

- Introdução à Segurança na Web
 - A implementação de práticas de segurança em aplicações web é fundamental para proteger dados sensíveis, garantir a integridade das operações e construir a confiança dos usuários.
 - Desde a adoção do protocolo HTTPS para garantir a segurança na transmissão de dados até a utilização de tokens JWT para autenticação e autorização eficientes, cada ponto destaca áreas específicas de preocupação e boas práticas.

Segurança

- Introdução à Segurança na Web
 - O gerenciamento seguro de sessões, a proteção contra injeções e a implementação de políticas de controle de acesso e autorização são aspectos críticos para prevenir ameaças conhecidas, enquanto a atualização regular de sistemas e a auditoria contínua fortalecem a resistência contra vulnerabilidades emergentes.



Segurança

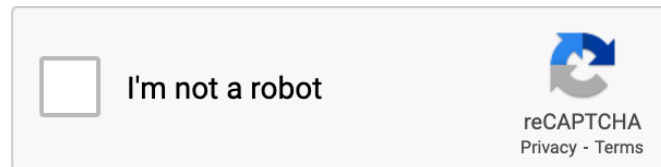
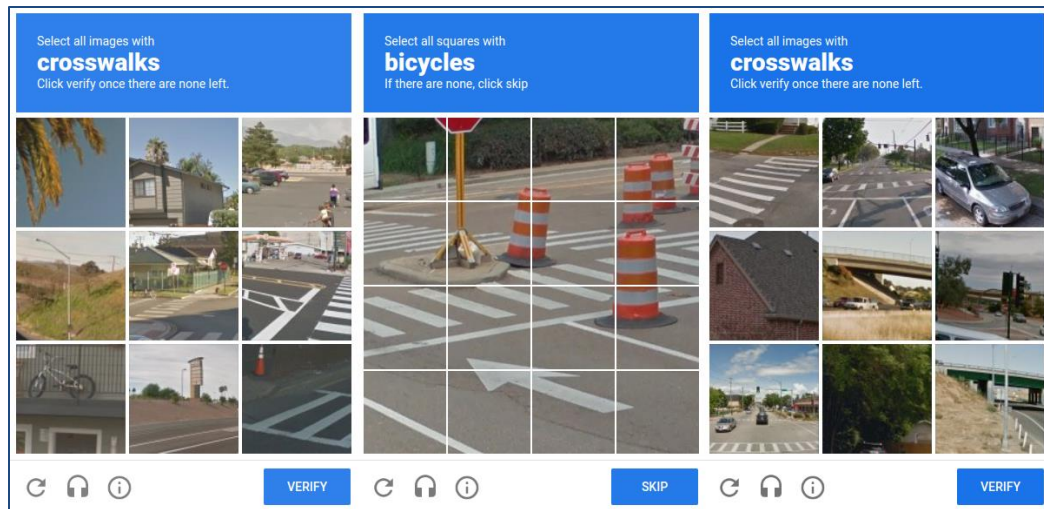
- Introdução à Segurança na Web
 - A conscientização e o treinamento contínuo de desenvolvedores e usuários, aliados a práticas como a criptografia adequada e a gestão segura de chaves, reforçam a postura de segurança de uma aplicação web.
 - A inclusão de desafios de reconhecimento humano, como CAPTCHAs, na aplicação web, fortalece a resistência contra atividades automatizadas maliciosas, garantindo a autenticidade das interações e protegendo pontos sensíveis contra exploração automatizada.

Segurança

- Introdução à Segurança na Web
 - A implementação de **CAPTCHAs** pode ajudar a prevenir ataques de força bruta, proteger contra registros automáticos indesejados, e reduzir o risco de abuso de funcionalidades que podem ser exploradas por bots maliciosos.
 - Além disso, eles são frequentemente usados para proteger formulários de contato, áreas de login e outros pontos de entrada críticos em aplicações web.

Segurança

- Introdução à Segurança na Web
 - CAPTCHAs:



Segurança

- Introdução à Segurança na Web
 - O **reCAPTCHA v3** do Google é projetado para ser invisível e não requer qualquer interação do usuário na forma de resolução de desafios ou CAPTCHAs visíveis.
 - Ao contrário das versões anteriores, onde os usuários eram solicitados a completar tarefas visíveis para provar que são humanos, o reCAPTCHA v3 opera de forma mais transparente.



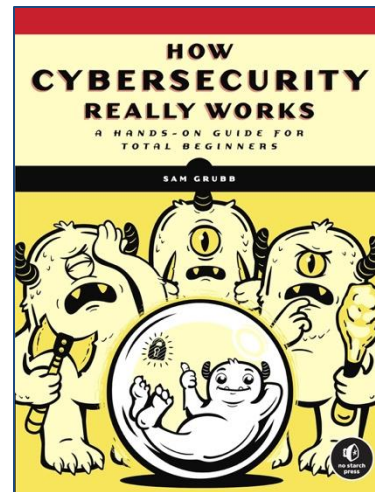
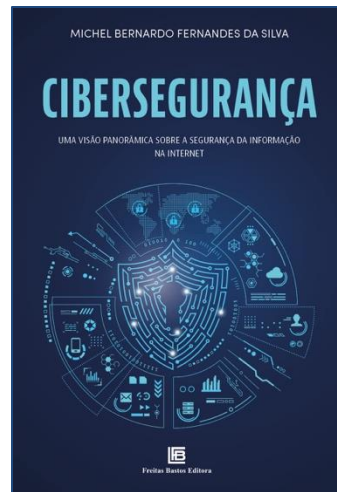
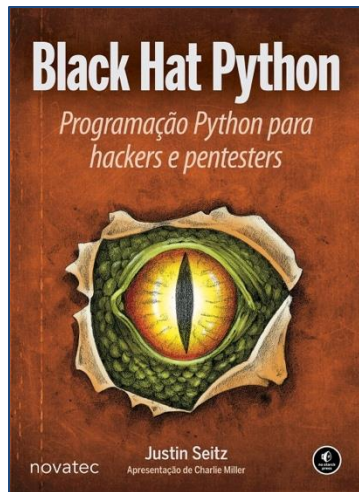
reCAPTCHA v3

Segurança

- Introdução à Segurança na Web
 - O reCAPTCHA v3 utiliza técnicas avançadas de análise de comportamento do usuário e machine learning para avaliar o risco de uma interação ser realizada por um bot ou um usuário legítimo.
 - Ele atribui uma pontuação de risco para cada interação, e você, como desenvolvedor, pode tomar decisões com base nessa pontuação, como bloquear interações de alto risco ou aplicar verificações adicionais.

Segurança

- Introdução à Segurança na Web
 - Alguns livros e leituras recomendadas sobre cibersegurança:



Sumário

- Segurança
 - Introdução à Segurança na Web
 - **JSON Web Token (JWT)**
 - Autenticação e Autorização

Segurança

- JSON Web Token (JWT)
 - A implementação de práticas específicas para garantir a segurança dos tokens JWT destaca sua relevância fundamental na autenticação, assegurando a integridade das comunicações e fortalecendo a segurança do sistema como um todo.
 - Antes de aprofundarmos nestas **práticas**, vejamos primeiro o que é um **token**.

Segurança

- O que é um **token**:
 - Em geral, um token é uma representação opaca ou simbólica de alguma autorização específica que foi concedida.
 - Em contextos de segurança e autenticação, tokens são frequentemente usados para provar que um usuário tem permissão para realizar determinadas ações ou acessar recursos específicos.

Segurança

- O que é um **token**:
 - Eles são emitidos por um sistema de autenticação após uma bem-sucedida autenticação do usuário e são utilizados para validar a identidade e as permissões do usuário em solicitações subsequentes.
 - Esses tokens são essenciais em ambientes distribuídos, como em aplicações web, onde a comunicação entre diferentes partes do sistema ocorre de forma desacoplada.

Segurança

- O que é um **token**:
 - Ao receber um token válido, o sistema pode confiar na autorização associada a esse token sem precisar consultar constantemente o sistema de autenticação. Isso não apenas melhora o desempenho, mas também reduz a dependência de comunicações frequentes com o servidor de autenticação.
 - A opacidade do token significa que os detalhes específicos da autorização não são diretamente visíveis para quem está interagindo com o token, proporcionando uma camada adicional de segurança.

Segurança

- O que é um **token**:
 - Além disso, os tokens podem ser configurados com prazos de expiração, incentivando a renovação periódica e contribuindo para uma abordagem mais dinâmica na gestão de sessões e permissões.
- No contexto das aplicações web modernas, os tokens são frequentemente utilizados em esquemas como **OAuth 2.0** para conceder acesso a recursos protegidos, como dados do usuário ou serviços.

Segurança

- O que é um **token**:
 - **OAuth 2.0** (Open Authorization 2.0) é um protocolo de autorização amplamente utilizado para delegação segura de acesso a recursos protegidos em aplicações web. Ele fornece um padrão para que usuários possam autorizar aplicativos de terceiros a acessar recursos em seus nomes sem compartilhar suas credenciais originais (como nome de usuário e senha).
- <https://docs.spring.io/spring-security/reference/servlet/oauth2/>
- <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>

Segurança

- O que é um **token**:
 - Existem também os tokens físicos, como pen drives (ou USB tokens), que são dispositivos de hardware que desempenham um papel significativo na segurança da informação, especialmente no contexto de autenticação e controle de acesso.
 - Esses tokens são projetados para armazenar chaves criptográficas, certificados digitais ou outras informações sensíveis e são usados como uma forma de garantir a identidade e autenticidade de um usuário ou dispositivo.

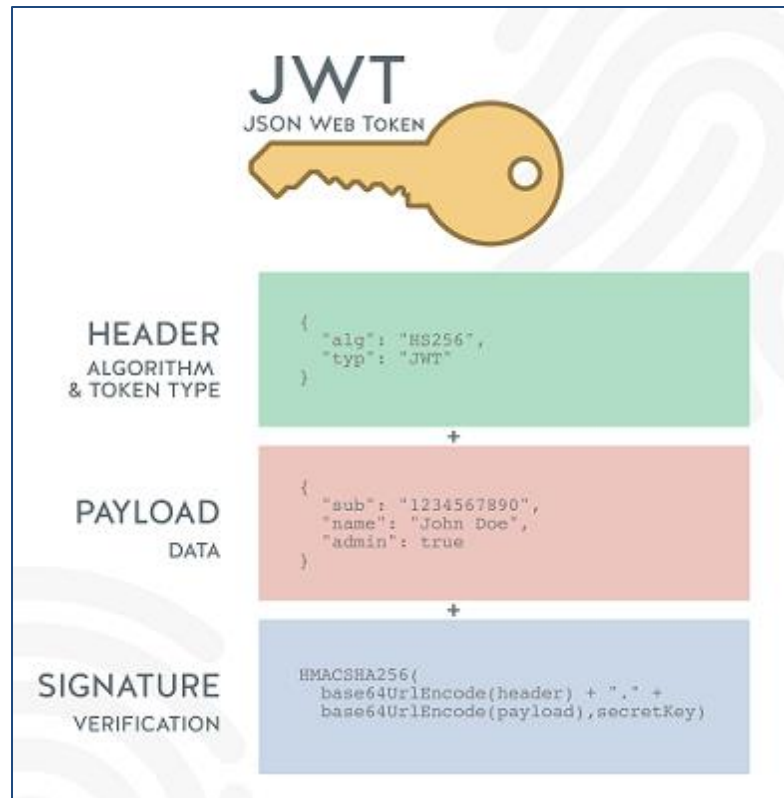
Segurança

- O que é um **token**:
 - É importante observar que, apesar das vantagens, tokens físicos também apresentam desafios, como a possibilidade de perda ou roubo.
 - Exemplos de tokens físicos que não exigem conexão à um computador:



Segurança

- JSON Web Token (JWT)
 - Os Tokens JWT são um tipo específico de token utilizado em aplicações web e serviços para autenticação e autorização.
 - Eles são uma forma compacta e autossuficiente de representar informações entre duas partes.
 - Os tokens JWT são frequentemente utilizados para transmitir informações sobre um usuário autenticado de uma forma segura entre o cliente e o servidor.

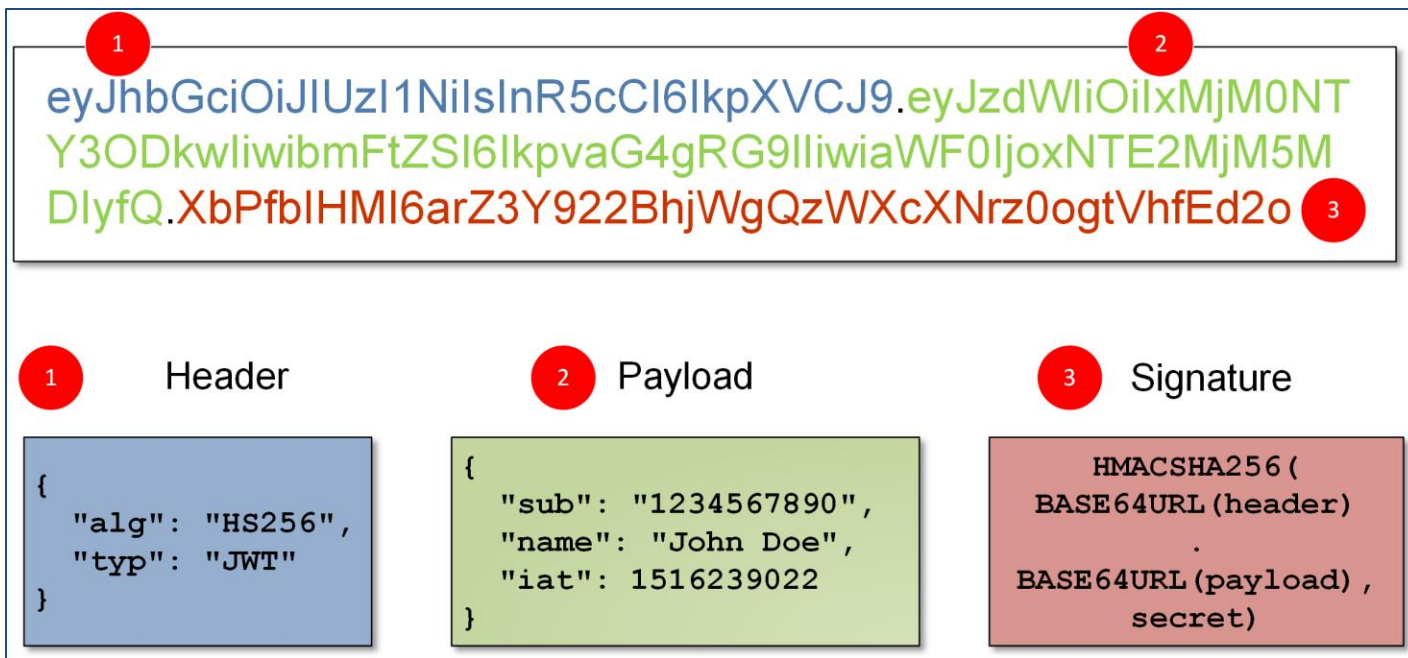


Segurança

- Um token JWT consiste em três partes separadas por pontos (.):
 - **Cabeçalho (Header):** Contém informações sobre como o token é processado e quais algoritmos criptográficos foram usados para assiná-lo.
 - **Carga Útil (Payload):** Contém as chamadas "claims" (afirmações), que são informações sobre a entidade (geralmente o usuário) e metadados adicionais. As claims podem incluir informações como ID do usuário, roles (papéis), expiração do token, entre outras.
 - **Assinatura (Signature):** Utilizada para verificar a integridade do token e garantir que não foi alterado durante a transmissão. A assinatura é gerada usando uma **chave secreta** conhecida apenas pelo servidor que emitiu o token.

Segurança

- JSON Web Token (JWT)



- JSON Web Token (JWT)
 - O JWT utiliza o formato JSON (JavaScript Object Notation) por várias razões, alinhadas com os benefícios e requisitos específicos de aplicações web modernas:
 - Estrutura simples e leve.
 - Facilidade de leitura e depuração.
 - Integração com linguagens de programação.
 - Compatibilidade com JavaScript.
 - Interoperabilidade e conformidade com padrões Web.
 - Extensibilidade e flexibilidade.
 - Suporte a tipos de dados diversificados.

Segurança

- JSON Web Token (JWT)
 - Os tokens JWT são frequentemente utilizados em cenários de autenticação *stateless*, onde o servidor não armazena informações de estado do usuário.
 - Isso significa que cada solicitação do cliente pode ser autenticada independentemente, tornando a arquitetura mais escalável e distribuída.
 - Eles são amplamente adotados em protocolos como **OAuth 2.0** e são escolhas comuns para sistemas modernos baseados em **microserviços**.

Segurança

- JSON Web Token (JWT)
 - Os tokens JWT são estruturas compactas e autossuficientes que podem transportar informações de forma segura entre partes confiáveis.
 - Eles são especialmente úteis para autenticação sem a necessidade de armazenar informações do usuário no servidor, permitindo assim uma abordagem *stateless*.



Segurança

- Práticas específicas de segurança relacionadas aos tokens JWT:
 - **Renovação de tokens e expiração adequada:** Implementar políticas de expiração para os tokens JWT e incluir mecanismos de renovação para evitar o uso indevido de tokens expirados. Isso reduz a janela de oportunidade para possíveis ataques.
 - **Assinatura digital e integridade:** Garantir a assinatura digital dos tokens JWT para verificar a integridade e autenticidade dos dados contidos. Isso protege contra tentativas de adulteração dos tokens.

Segurança

- Práticas específicas de segurança relacionadas aos tokens JWT:
 - **Claims minimizados e seguros:** Limitar as informações sensíveis (claims) incluídas nos tokens JWT apenas ao necessário. Evitar incluir dados sensíveis, como senhas, e garantir que as informações armazenadas não comprometam a segurança.
 - **Gerenciamento seguro de chaves secretas:** Armazenar e gerenciar as chaves secretas utilizadas para assinar os tokens JWT de forma segura, evitando exposição indevida. Considere o uso de soluções de gerenciamento de segredos.

Segurança

- Práticas específicas de segurança relacionadas aos tokens JWT:
 - **Uso adequado de algoritmos criptográficos:** Escolher algoritmos de assinatura e criptografia robustos e atualizados. Evitar algoritmos desaconselhados ou considerados inseguros.
 - **HTTPS para transmissão de tokens:** Utilizar HTTPS para a transmissão segura de tokens JWT entre o cliente e o servidor, protegendo contra interceptação e manipulação durante a transferência.

Segurança

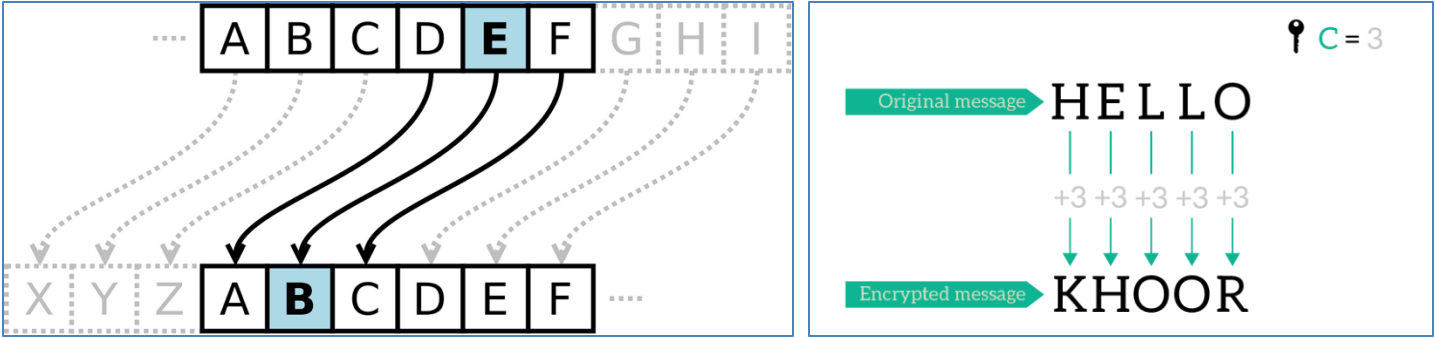
- **Algoritmos criptográficos:**

- Algoritmos criptográficos são conjuntos de procedimentos matemáticos e lógicos que desempenham um papel fundamental na proteção da informação sensível por meio de técnicas de criptografia.
- Esses algoritmos são projetados para garantir a confidencialidade, integridade e autenticidade dos dados, utilizando diferentes abordagens, como cifras simétricas e assimétricas, funções de hash e assinaturas digitais.

Segurança

- **Algoritmos criptográficos:**
 - Enquanto as cifras simétricas usam uma chave única para cifrar e decifrar dados, as cifras assimétricas utilizam pares de chaves públicas e privadas, oferecendo métodos mais seguros de comunicação e autenticação.
 - Os algoritmos criptográficos desempenham um papel crucial em diversos cenários, desde a proteção de transações online até a garantia da privacidade das comunicações digitais, contribuindo para a construção de sistemas seguros e confiáveis em ambientes digitais.

- **Algoritmos criptográficos:**
 - A Cifra de César é um exemplo de algoritmo criptográfico simétrico, onde uma única chave é utilizada tanto para cifrar quanto para decifrar as informações.
 - O nome do método é em homenagem a Júlio César, que o usou para se comunicar com os seus generais durante o Império Romano.



- **Algoritmos criptográficos:**
 - A máquina **Enigma**, usada pela Alemanha Nazista durante a Segunda Guerra Mundial, era um dispositivo de criptografia simétrica.
 - A Enigma foi quebrada pela primeira vez por Alan Mathison Turing (1912-1954) e sua equipe.



- Algoritmos criptográficos:
 - Veja o filme “The Imitation Game” (2014).
 - “O Jogo da Imitação”:
 - <https://www.imdb.com/title/tt2084970/>
 - Conheça o “Prêmio Alan Turing”:
 - <https://amturing.acm.org/>
 - Considerado o prêmio Nobel da computação.
 - Dificilmente é dado à alguém.
 - Conheça o “Teste de Turing”:
 - <https://goo.gl/pp2T6r>



- **Algoritmos criptográficos:**

- Os tokens JWT suportam diversos algoritmos criptográficos para assinatura e criptografia. Aqui estão alguns dos algoritmos mais comuns utilizados:

- **HS256** (HMAC com SHA-256):

- **Descrição:** Este algoritmo usa uma chave secreta compartilhada para gerar uma assinatura HMAC (Hash-based Message Authentication Code) usando a função de hash SHA-256. É um método simétrico, o que significa que a mesma chave é usada para assinar e verificar a assinatura.
- **Uso:** Adequado para situações em que há uma relação de confiança entre o emissor e o receptor do token.



- **Algoritmos criptográficos:**
 - Os tokens JWT suportam diversos algoritmos criptográficos para assinatura e criptografia. Aqui estão alguns dos algoritmos mais comuns utilizados:
 - **HS512** (HMAC com SHA-512): [Usaremos este no projeto desta aula]
 - **Descrição:** Este algoritmo utiliza uma chave secreta compartilhada para gerar uma assinatura HMAC (Hash-based Message Authentication Code) usando a função de hash SHA-512. Da mesma forma que o HS256, é um método simétrico.
 - **Uso:** Recomendado para situações em que há uma relação de confiança entre o emissor e o receptor do token, similar ao HS256. Geralmente, algoritmos com tamanhos de hash mais longos, como o SHA-512, são considerados mais seguros, mas também podem demandar mais recursos computacionais.



- **Algoritmos criptográficos:**

- Os tokens JWT suportam diversos algoritmos criptográficos para assinatura e criptografia. Aqui estão alguns dos algoritmos mais comuns utilizados:

- **RS256** (RSA com SHA-256):

- **Descrição:** O emissor usa uma chave privada para assinar o token, e o receptor utiliza a chave pública correspondente para verificar a assinatura. É um método assimétrico, proporcionando uma maior segurança em ambientes onde chaves públicas e privadas podem ser utilizadas separadamente.
- **Uso:** Recomendado para situações em que a chave privada permanece segura com o emissor, enquanto a chave pública pode ser distribuída para validação.



- **Algoritmos criptográficos:**

- Os tokens JWT suportam diversos algoritmos criptográficos para assinatura e criptografia. Aqui estão alguns dos algoritmos mais comuns utilizados:

- **ES256** (ECDSA com SHA-256):

- **Descrição:** Utiliza a criptografia de curva elíptica (ECDSA) para gerar a assinatura. Similar ao RS256, é um método assimétrico, mas utiliza chaves mais curtas, proporcionando eficiência computacional.
- **Uso:** É uma boa escolha para ambientes com restrições de recursos computacionais.



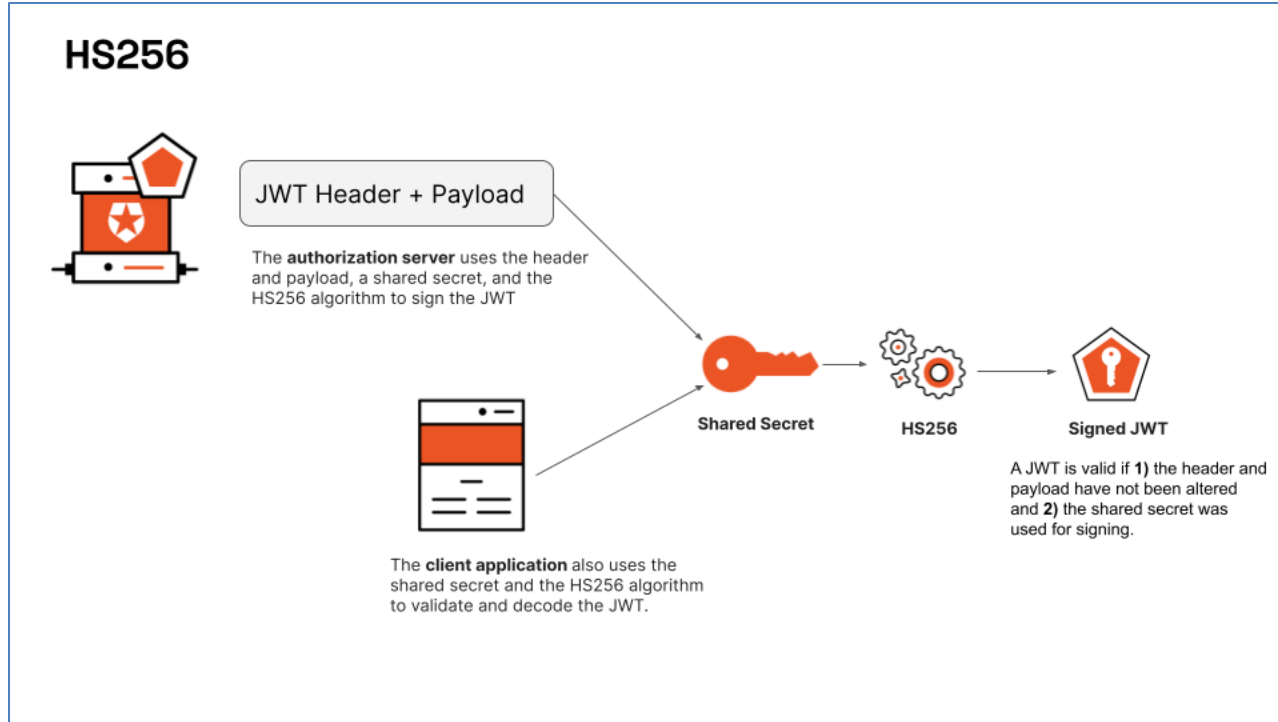
Segurança

- **Algoritmos criptográficos:**

- A escolha do algoritmo depende do contexto específico, requisitos de segurança, e considerações de desempenho.
- É crucial selecionar o algoritmo adequado com base nas necessidades da aplicação e nas práticas recomendadas de segurança.
- Leitura sugerida:
 - <https://community.auth0.com/t/rs256-vs-hs256-jwt-signing-algorithms/58609>

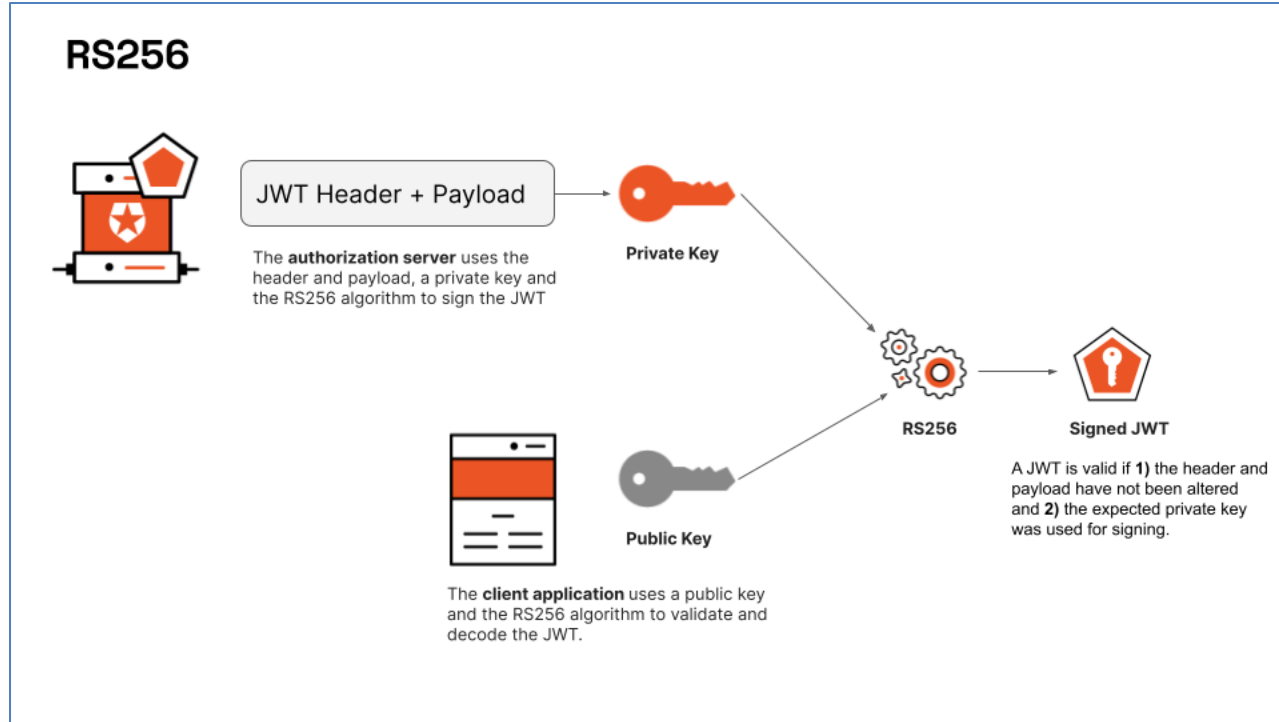
Segurança

- Algoritmos criptográficos:



Segurança

- Algoritmos criptográficos:



Sumário

- Segurança
 - Introdução à Segurança na Web
 - JSON Web Token (JWT)
 - **Autenticação e Autorização**

- Autenticação e Autorização
 - **Autenticação:**
 - **Definição:** A autenticação refere-se ao processo de verificar a identidade de um usuário, dispositivo ou sistema. É o ato de confirmar que a entidade que está solicitando acesso é realmente quem ela alega ser.
 - **Métodos:** Pode envolver o uso de credenciais como nome de usuário e senha, autenticação biométrica (por exemplo, impressões digitais, reconhecimento facial), chaves criptográficas, tokens, entre outros.
 - **Objetivo:** Garantir que apenas usuários legítimos tenham acesso aos recursos ou serviços protegidos.



- Autenticação e Autorização

- **Autorização:**

- **Definição:** A autorização é o processo de conceder ou negar permissões específicas a um usuário ou sistema autenticado. Depois que a autenticação é realizada com sucesso, a autorização determina quais ações ou recursos o usuário tem permissão para acessar.
- **Métodos:** Baseia-se nas políticas de controle de acesso, que definem as regras sobre quem pode fazer o quê. As permissões podem ser atribuídas a níveis granulares, como leitura, gravação, execução, etc.
- **Objetivo:** Garantir que usuários autenticados tenham acesso apenas às informações e operações para as quais têm permissão.



Segurança

- **Autenticação vs Autorização:**

- Autenticação e autorização são conceitos distintos em segurança de sistemas e controle de acesso.
- A autenticação verifica a identidade, enquanto a autorização controla o acesso com base nas permissões atribuídas a uma identidade autenticada.
- Esses dois conceitos trabalham em conjunto para garantir que apenas usuários legítimos possam realizar ações apropriadas em um sistema ou aplicação.

Segurança

Autenticação



SIGN IN



USERNAME



PASSWORD

LOGIN



Remember Me

[Forgot Password?](#)

Autorização



CONTROLE DE ACESSO



123.456.789-10



Maria Antônia Silva Santos



Financeiro

Contas a Pagar



Contas a Receber



Controle de Estoque



Cadastro de Usuários



Cadastro de Clientes



Vendas



Pedidos



Vamos à prática!

- Vimos então que JSON Web Tokens (JWT) se tornaram um método popular para proteger aplicações web modernas.
- Os JWTs permitem transmitir informações com segurança entre as partes como um objeto JSON compacto, independente e assinado digitalmente.
- Crie uma API REST que implemente a autenticação JWT em uma aplicação Spring Boot.

- Primeiramente, crie a aplicação Spring Boot:
 - <https://start.spring.io/>

 **spring** inicializ

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Java

☐ Kotlin

☐ Groovy

☒ Maven

Spring Boot

☐ 3.3.0 (SNAPSHOT)

☐ 3.2.2 (SNAPSHOT)

☒ 3.2.1

☐ 3.1.8 (SNAPSHOT)

☐ 3.1.7

Project Metadata

Group

com.example

Artifact

JWT_RestAPI

Name

JWT_RestAPI

Description

JWT_RestAPI com Spring Boot

Package name

com.example.JWT_RestAPI

Packaging

☒ Jar

☐ War

Java

☐ 21

☒ 17

Dependencies

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Security

SECURITY

Highly customizable authentication and access-control framework for Spring applications.

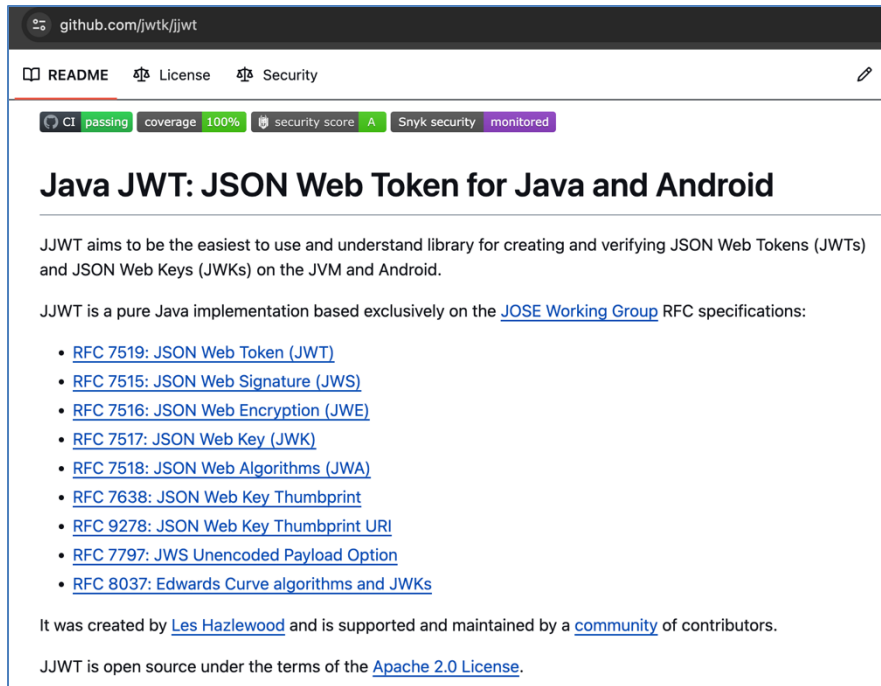
ADD DEPENDENCIES... ⌘ + B

GENERATE ⌘ + ↵

EXPLORE CTRL + SPACE

SHARE...

- Utilizaremos também a biblioteca open source **jjwt**:
 - Java JWT: JSON Web Token for Java and Android
 - <https://github.com/jwtk/jjwt>



The screenshot shows the GitHub repository page for `github.com/jwtk/jjwt`. The page header includes links for `README`, `License`, and `Security`. Below the header, there are status badges: `CI passing`, `coverage 100%`, `security score A`, and `Snyk security monitored`. The main heading is **Java JWT: JSON Web Token for Java and Android**. The text describes the library as the easiest to use and understand for creating and verifying JSON Web Tokens (JWTs) and JSON Web Keys (JWKs) on the JVM and Android. It mentions that jjwt is a pure Java implementation based on the [JOSE Working Group](#) RFC specifications. A list of RFCs is provided: [RFC 7519: JSON Web Token \(JWT\)](#), [RFC 7515: JSON Web Signature \(JWS\)](#), [RFC 7516: JSON Web Encryption \(JWE\)](#), [RFC 7517: JSON Web Key \(JWK\)](#), [RFC 7518: JSON Web Algorithms \(JWA\)](#), [RFC 7638: JSON Web Key Thumbprint](#), [RFC 9278: JSON Web Key Thumbprint URI](#), [RFC 7797: JWS Unencoded Payload Option](#), and [RFC 8037: Edwards Curve algorithms and JWKs](#). It also states that the library was created by [Les Hazlewood](#) and is supported and maintained by a [community](#) of contributors. Finally, it mentions that jjwt is open source under the terms of the [Apache 2.0 License](#).



```
19 <dependencies>
20   <!-- Spring Boot Starter -->
21   <dependency>
22     <groupId>org.springframework.boot</groupId>
23     <artifactId>spring-boot-starter-web</artifactId>
24   </dependency>
25   <dependency>
26     <groupId>org.springframework.boot</groupId>
27     <artifactId>spring-boot-starter-test</artifactId>
28     <scope>test</scope>
29   </dependency>
30   <!-- Spring Security -->
31   <dependency>
32     <groupId>org.springframework.boot</groupId>
33     <artifactId>spring-boot-starter-security</artifactId>
34   </dependency>
35   <dependency>
36     <groupId>org.springframework.security</groupId>
37     <artifactId>spring-security-test</artifactId>
38     <scope>test</scope>
39   </dependency>
```

Spring Boot Starter



Spring Security



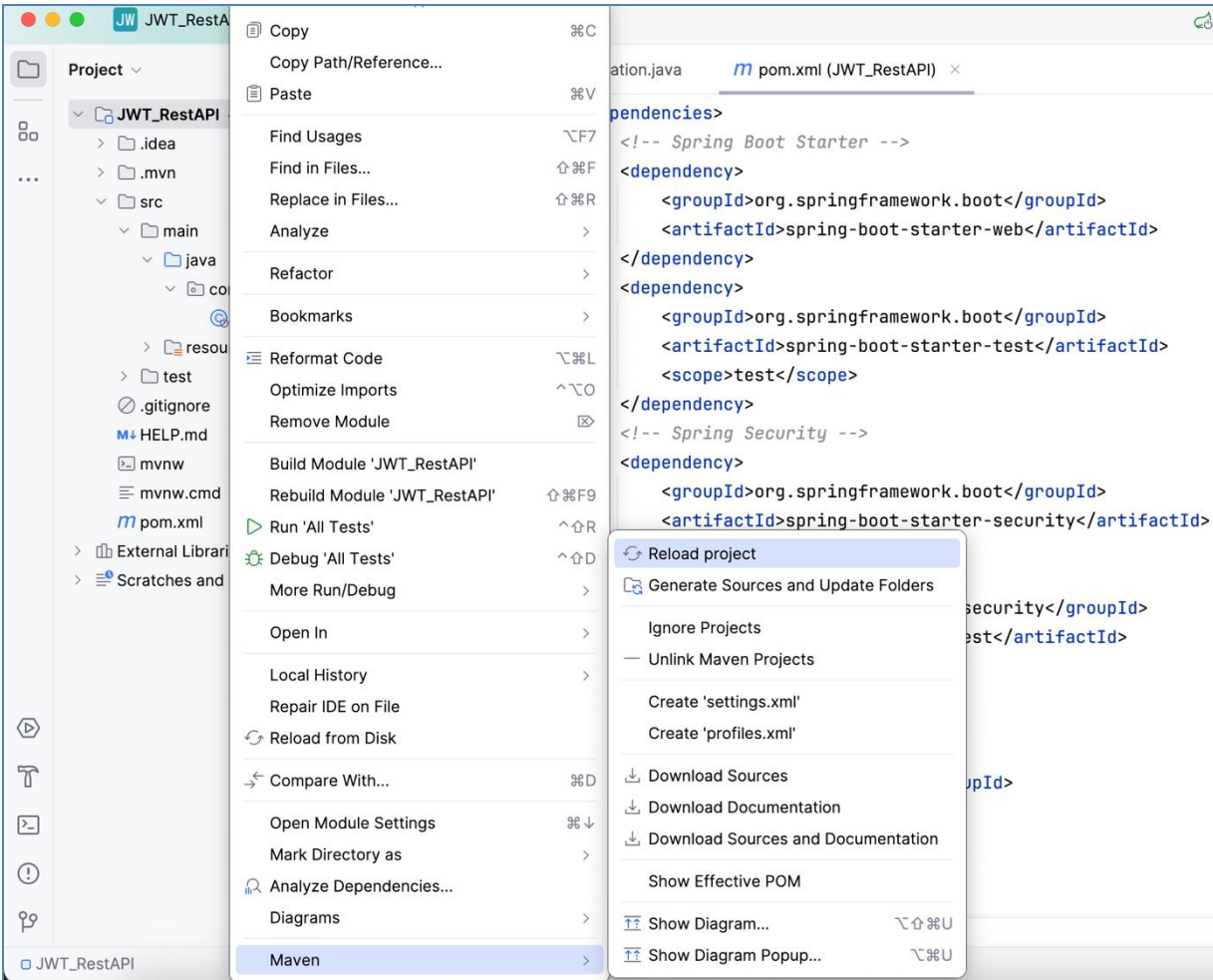
- Vejamos acima como ficam as nossas dependências no arquivo de configuração **pom.xml**.

```
40  <!-- JWT Library -->
41  <dependency>
42      <groupId>io.jsonwebtoken</groupId>
43      <artifactId>jjwt-api</artifactId>
44      <version>0.12.3</version>
45  </dependency>
46  <dependency>
47      <groupId>io.jsonwebtoken</groupId>
48      <artifactId>jjwt-impl</artifactId>
49      <version>0.12.3</version>
50      <scope>runtime</scope>
51  </dependency>
52  <dependency>
53      <groupId>io.jsonwebtoken</groupId>
54      <artifactId>jjwt-gson</artifactId>
55      <version>0.12.3</version>
56      <scope>runtime</scope>
57  </dependency>
58  </dependencies>
```

JWT Library



- Vejamos acima como ficam as nossas dependências no arquivo de configuração **pom.xml**.



- Lembre-se de clicar com o botão direito no projeto e ir em: **Maven > Reload project.**
- Essa opção permite recarregar ou atualizar as configurações e dependências do projeto Maven.

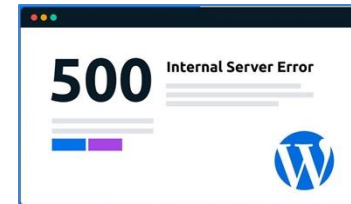
- Antes de seguirmos com a prática, precisamos entender o significado dos principais códigos de resposta utilizados por servidores web para indicar o status de uma solicitação HTTP.
 - **200 OK:** Indica que a solicitação foi bem-sucedida. O servidor respondeu com sucesso à requisição do cliente, e a informação solicitada está no corpo da resposta.
 - **401 Unauthorized:** Significa que a requisição não foi autorizada. O cliente deve fornecer credenciais válidas (por exemplo, nome de usuário e senha) para acessar os recursos solicitados.



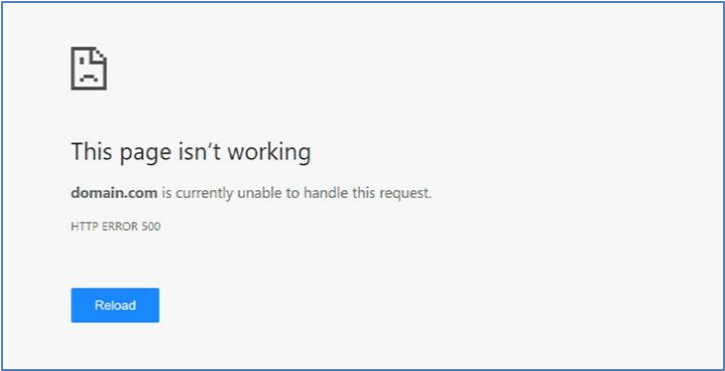
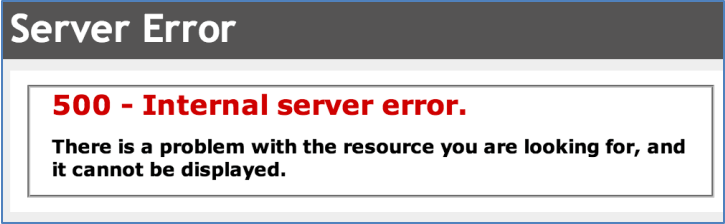
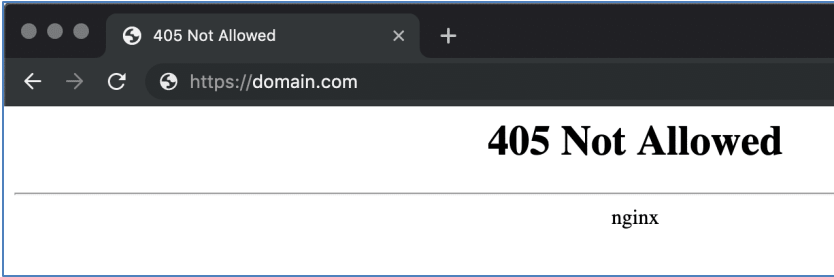
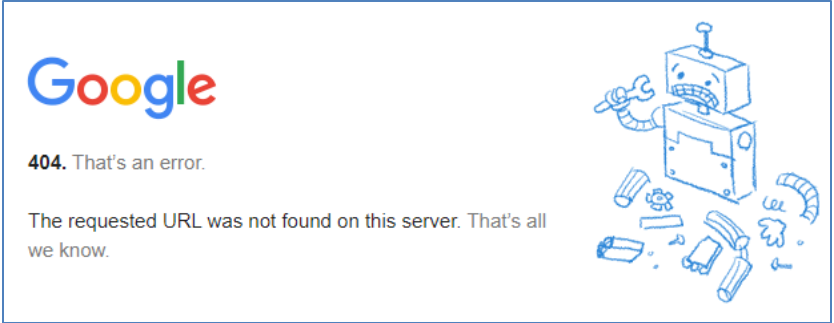
- Antes de seguirmos com a prática, precisamos entender o significado dos principais códigos de resposta utilizados por servidores web para indicar o status de uma solicitação HTTP.
 - **403 Forbidden:** Indica que o servidor entende a solicitação, mas o acesso ao recurso é proibido. Diferentemente do código 401, a autenticação não fará diferença e não autorizará o acesso.
 - **404 Not found:** Sinaliza que o recurso solicitado não foi encontrado no servidor. Isso pode ocorrer quando a URL está incorreta, o recurso foi removido ou não está disponível no momento.



- Antes de seguirmos com a prática, precisamos entender o significado dos principais códigos de resposta utilizados por servidores web para indicar o status de uma solicitação HTTP.
 - **405 Method not allowed:** Indica que o método de requisição (GET, POST, PUT, etc.) utilizado não é permitido para o recurso solicitado.
 - **500 Internal Server Error:** Este é um código genérico para uma mensagem de erro no servidor. Indica que algo deu errado no lado do servidor, mas não especifica a natureza do problema. Pode ser um erro temporário ou um problema mais grave no servidor.



- Esses códigos são parte do protocolo HTTP e são úteis para os desenvolvedores entenderem o resultado de suas solicitações e diagnosticar problemas em suas aplicações web.



- Exemplo de Internal Server Error – Status Code 500:
 - É comum vir acompanhado de uma Exception.

HTTP Status [500] – [Internal Server Error]

Type Exception Report

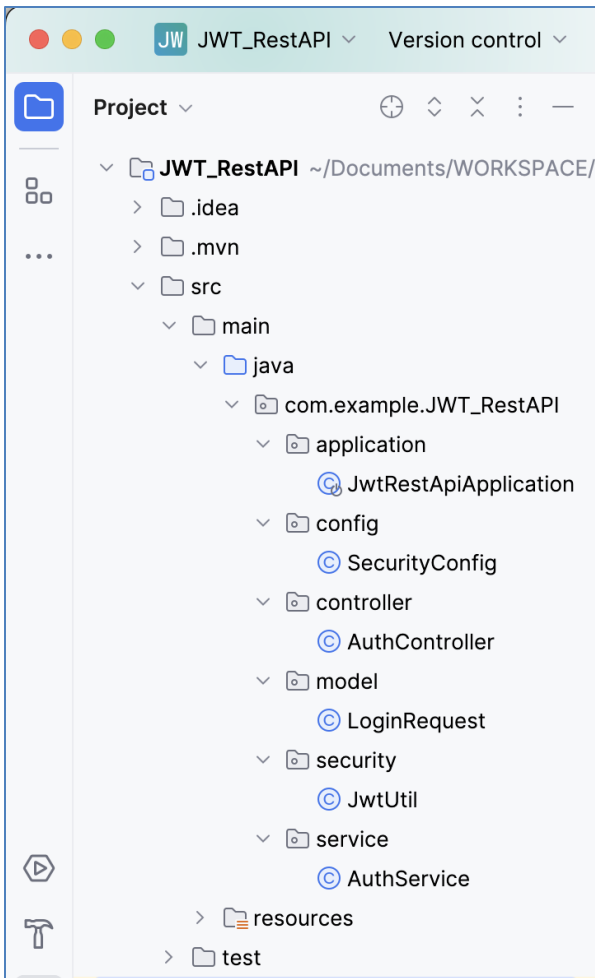
Message Handler dispatch failed; nested exception is java.lang.StackOverflowError

Description The server encountered an unexpected condition that prevented it from fulfilling the request.

Exception

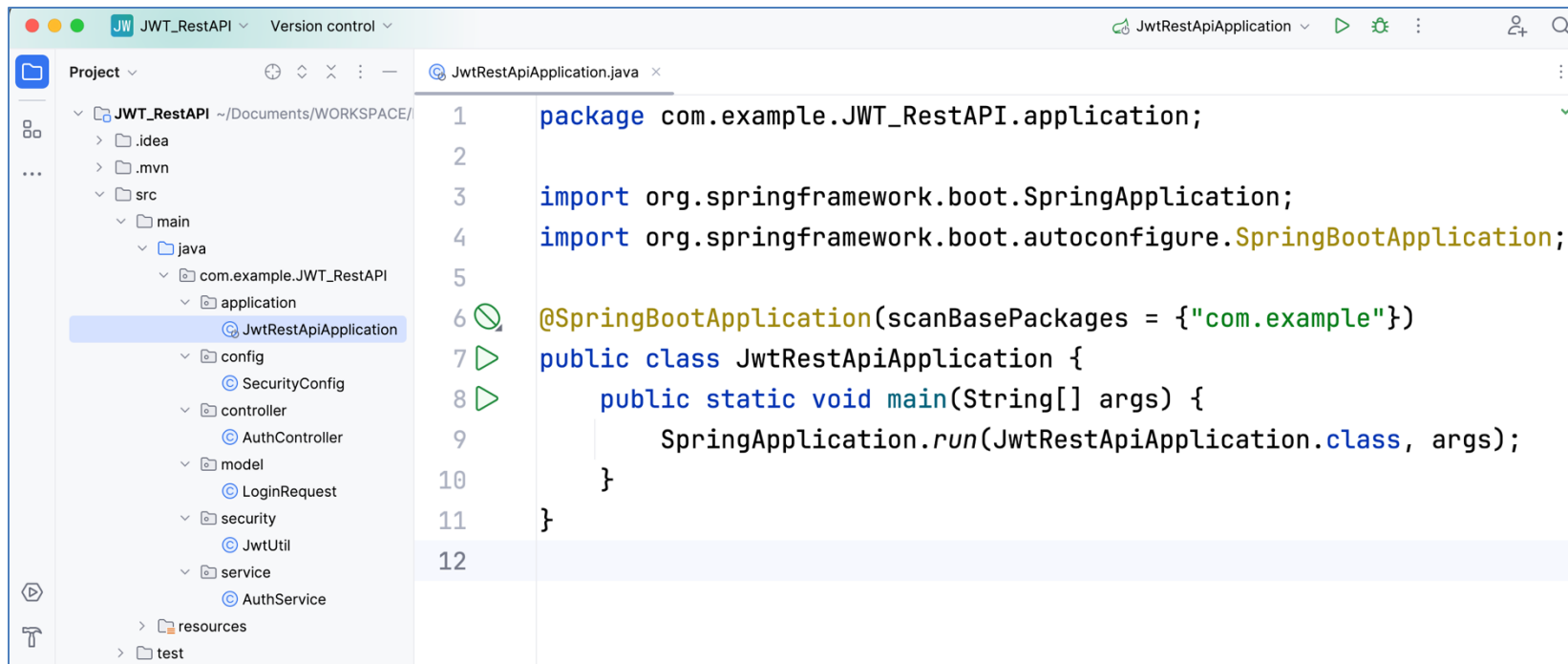
```
org.springframework.web.util.NestedServletException: Handler dispatch failed; nested exception is java.lang.StackOverflowError
    org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:982)
    org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:901)
    org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:970)
    org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.java:861)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:635)
    org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:846)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:742)
    org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:52)
    org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:99)
    org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:107)
    org.springframework.web.filter.HttpPutFormContentFilter.doFilterInternal(HttpPutFormContentFilter.java:105)
    org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:107)
    org.springframework.web.filter.HiddenHttpMethodFilter.doFilterInternal(HiddenHttpMethodFilter.java:81)
    org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:107)
    org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:197)
    org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:107)
```

Root Cause



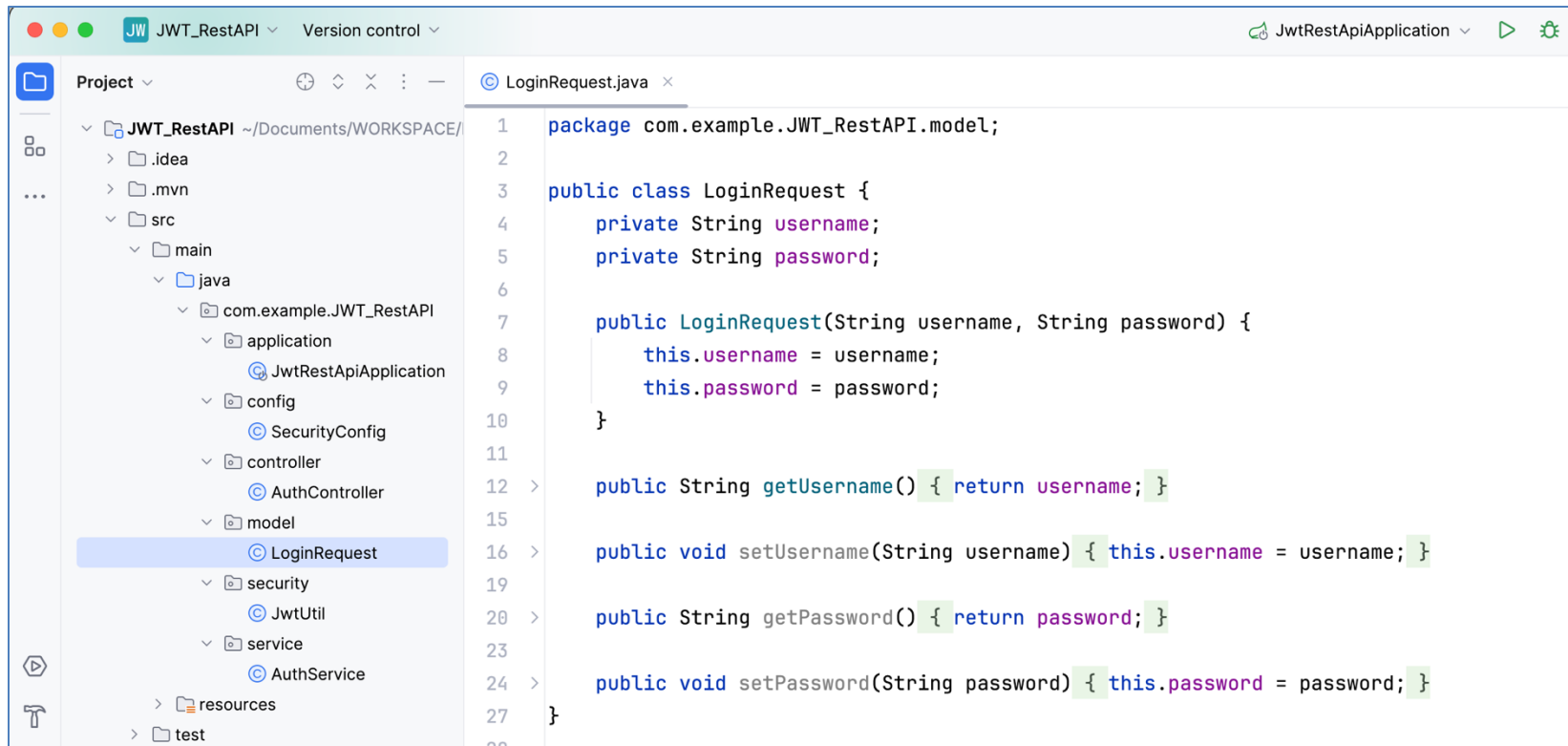
- Voltando à prática, veja agora como ficará a estrutura do projeto.
- Estamos seguindo os mesmos padrões de arquitetura das aulas anteriores. Temos os pacotes: **application**, **controller**, **model** e **service**. Somente o pacote **repository** não foi criado pois não há um banco de dados desta vez (sugestão: crie um BD MongoDB para o login e pratique).
- A novidade aqui será um pacote **config** e outro pacote **security**.

- Classe **JwtRestApiApplication** com o método main.
- Nada muda em relação aos projetos anteriores.



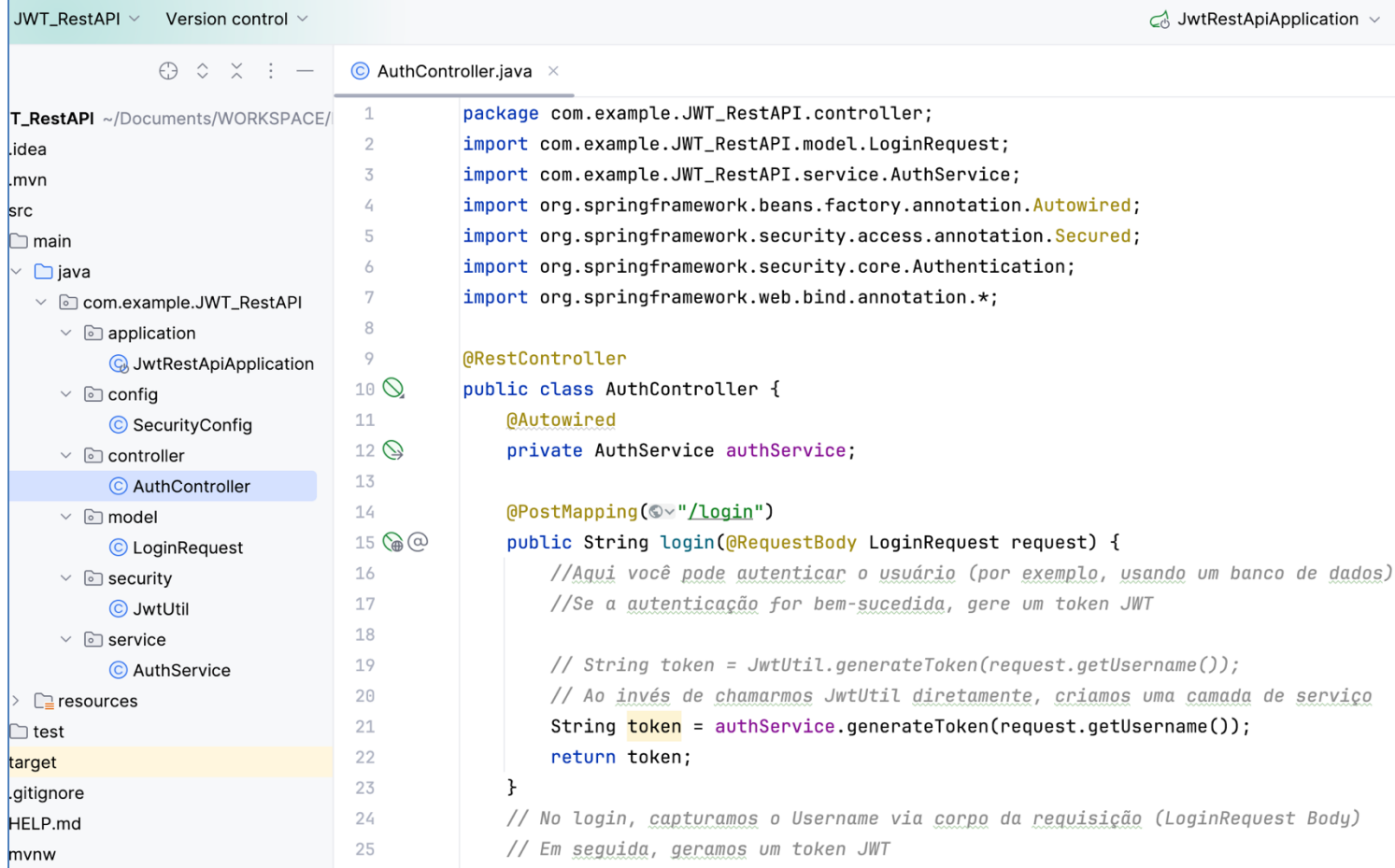
```
1 package com.example.JWT_RestAPI.application;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication(scanBasePackages = {"com.example"})
7 public class JwtRestApiApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(JwtRestApiApplication.class, args);
10    }
11 }
12
```

- Classe **LoginRequest**: Nosso modelo que representa o login.
 - Representa o corpo da requisição http (@RequestBody).



```
1 package com.example.JWT_RestAPI.model;
2
3 public class LoginRequest {
4     private String username;
5     private String password;
6
7     public LoginRequest(String username, String password) {
8         this.username = username;
9         this.password = password;
10    }
11
12    public String getUsername() { return username; }
13
14    public void setUsername(String username) { this.username = username; }
15
16    public String getPassword() { return password; }
17
18    public void setPassword(String password) { this.password = password; }
19 }
```

Classe **AuthController**
com os **endpoints** e as
chamadas para a classe
AuthService.



The screenshot shows an IDE with a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with the 'AuthController' class highlighted under the 'controller' package. The main editor window shows the code for 'AuthController.java'.

```
JWT_RestAPI ~\Documents\WORKSPACE\
idea
.mvn
src
  main
    java
      com.example.JWT_RestAPI
        application
          JwtRestApiApplication
        config
          SecurityConfig
        controller
          AuthController
        model
          LoginRequest
        security
          JwtUtil
        service
          AuthService
    resources
    test
target
.gitignore
HELP.md
mvnw
```

```
1 package com.example.JWT_RestAPI.controller;
2 import com.example.JWT_RestAPI.model.LoginRequest;
3 import com.example.JWT_RestAPI.service.AuthService;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.security.access.annotation.Secured;
6 import org.springframework.security.core.Authentication;
7 import org.springframework.web.bind.annotation.*;
8
9 @RestController
10 public class AuthController {
11     @Autowired
12     private AuthService authService;
13
14     @PostMapping("/login")
15     public String login(@RequestBody LoginRequest request) {
16         //Aqui você pode autenticar o usuário (por exemplo, usando um banco de dados)
17         //Se a autenticação for bem-sucedida, gere um token JWT
18
19         // String token = JwtUtil.generateToken(request.getUsername());
20         // Ao invés de chamarmos JwtUtil diretamente, criamos uma camada de serviço
21         String token = authService.generateToken(request.getUsername());
22         return token;
23     }
24     // No login, capturamos o Username via corpo da requisição (LoginRequest Body)
25     // Em seguida, geramos um token JWT
```

- Classe **AuthController** com os **endpoints** e as chamadas para a classe **AuthService**.

```
27 @GetMapping("/{username}/{token}")
28 public String extractUsername(@PathVariable String token) {
29     // String username = JwtUtil.extractUsername(token);
30     // Ao invés de chamarmos JwtUtil diretamente, utilizamos a camada de serviço
31     String username = authService.extractUsername(token);
32     return username;
33 }
34 // No extractUsername, capturamos o token via URL apenas por praticidade
35 // (poderia ser via @RequestBody também)
36 // Em seguida, extraímos o username deste token
37
38 // Qualquer um irá acessar
39 @GetMapping("/user")
40 public String getUser(Authentication authentication) { return "User: " + authentication.getName(); }
41
42
43
44 // Somente o admin irá acessar
45 @Secured("ADMIN")
46 @GetMapping("/admin")
47 public String onlyAdmin(Authentication authentication) { return "Admin: " + authentication.getName(); }
48
49
50 }
51
```

The screenshot shows an IDE window for a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with the 'AuthService' class highlighted under the 'service' package. The main editor area shows the code for 'AuthService.java'.

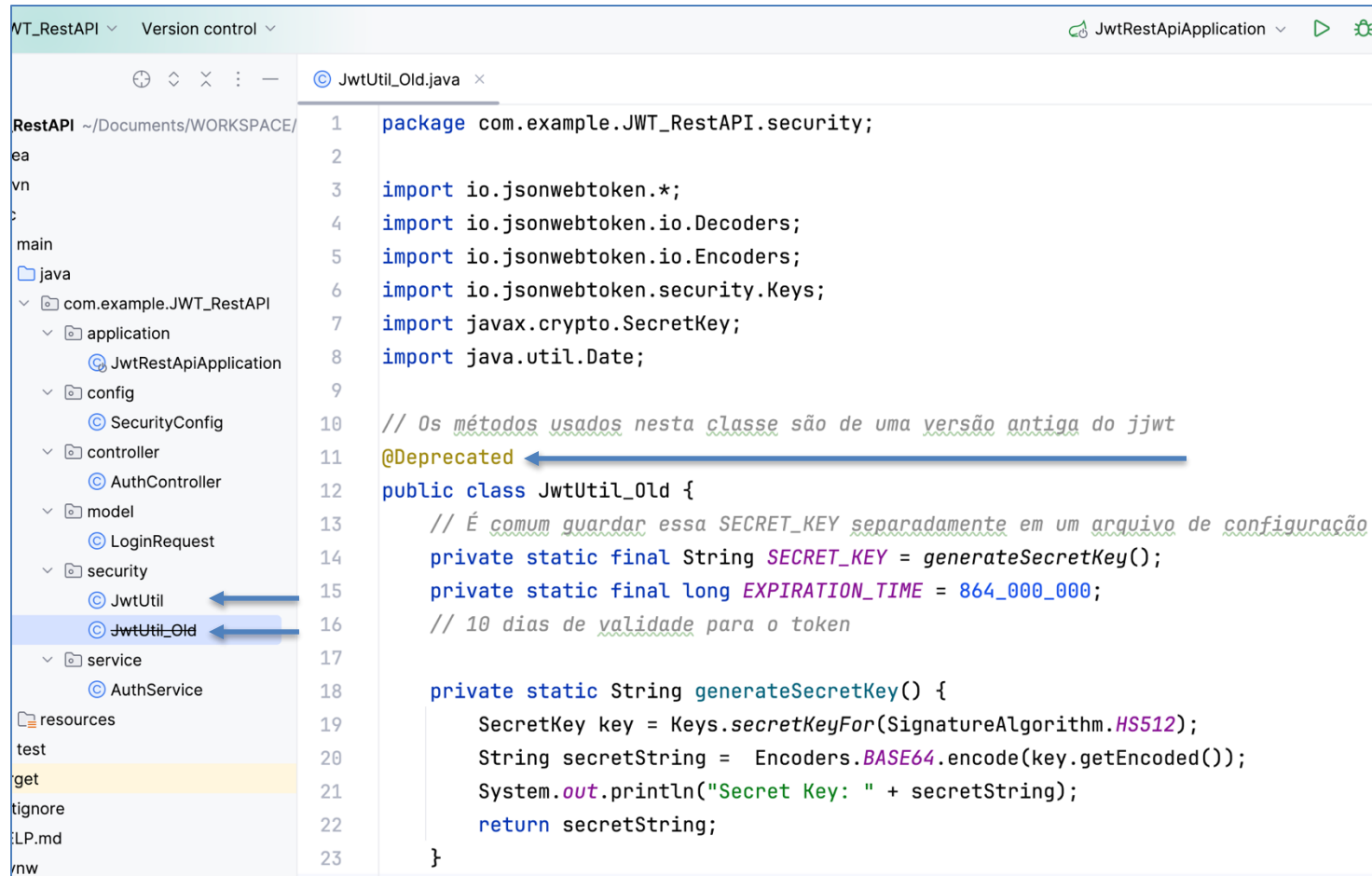
```
1 package com.example.JWT_RestAPI.service;
2
3 import com.example.JWT_RestAPI.security.JwtUtil;
4 import org.springframework.stereotype.Service;
5
6 @Service
7 public class AuthService {
8     public String generateToken(String username) {
9         String token = JwtUtil.generateToken(username);
10        return token;
11    }
12
13    public String extractUsername(String token) {
14        String username = JwtUtil.extractUsername(token);
15        return username;
16    }
17 }
18
```

Classe **AuthService** com os serviços e as chamadas para o utilitário de segurança.

As chamadas para um BD (em um pacote **repository**) também podem ficar aqui.

Classes ou métodos defasados, ou seja, de uma versão antiga ou legada de uma biblioteca (jjwt), ficam anotados com **@Deprecated** (depreciado).

Você pode comparar as classes **JwtUtil** e **JwtUtil_Old** (antiga) para ver a diferença.



```
WT_RestAPI  Version control  JwtRestApiApplication  JwtUtil_Old.java x

RestAPI ~/Documents/WORKSPACE/ 1 package com.example.JWT_RestAPI.security;
ea 2
vn 3 import io.jsonwebtoken.*;
c 4 import io.jsonwebtoken.io.Decoders;
main 5 import io.jsonwebtoken.io.Encoders;
java 6 import io.jsonwebtoken.security.Keys;
  com.example.JWT_RestAPI 7 import javax.crypto.SecretKey;
    application 8 import java.util.Date;
      JwtRestApiApplication 9
        config 10 // Os métodos usados nesta classe são de uma versão antiga do jjwt
          SecurityConfig 11 @Deprecated
        controller 12 public class JwtUtil_Old {
          AuthController 13 // É comum guardar essa SECRET_KEY separadamente em um arquivo de configuração
        model 14 private static final String SECRET_KEY = generateSecretKey();
          LoginRequest 15 private static final long EXPIRATION_TIME = 864_000_000;
        security 16 // 10 dias de validade para o token
          JwtUtil 17
          JwtUtil_Old 18 private static String generateSecretKey() {
        service 19     SecretKey key = Keys.secretKeyFor(SignatureAlgorithm.HS512);
          AuthService 20     String secretString = Encoders.BASE64.encode(key.getEncoded());
resources 21     System.out.println("Secret Key: " + secretString);
test 22     return secretString;
  get 23 }
  ignore
  LP.md
  nw
```

Classe **JwtUtil** com os métodos utilitários de segurança.

Foi atualizada para utilizar os métodos da versão 0.12.3 do jjwt.

Classe **JwtUtil** com os métodos utilitários de segurança.

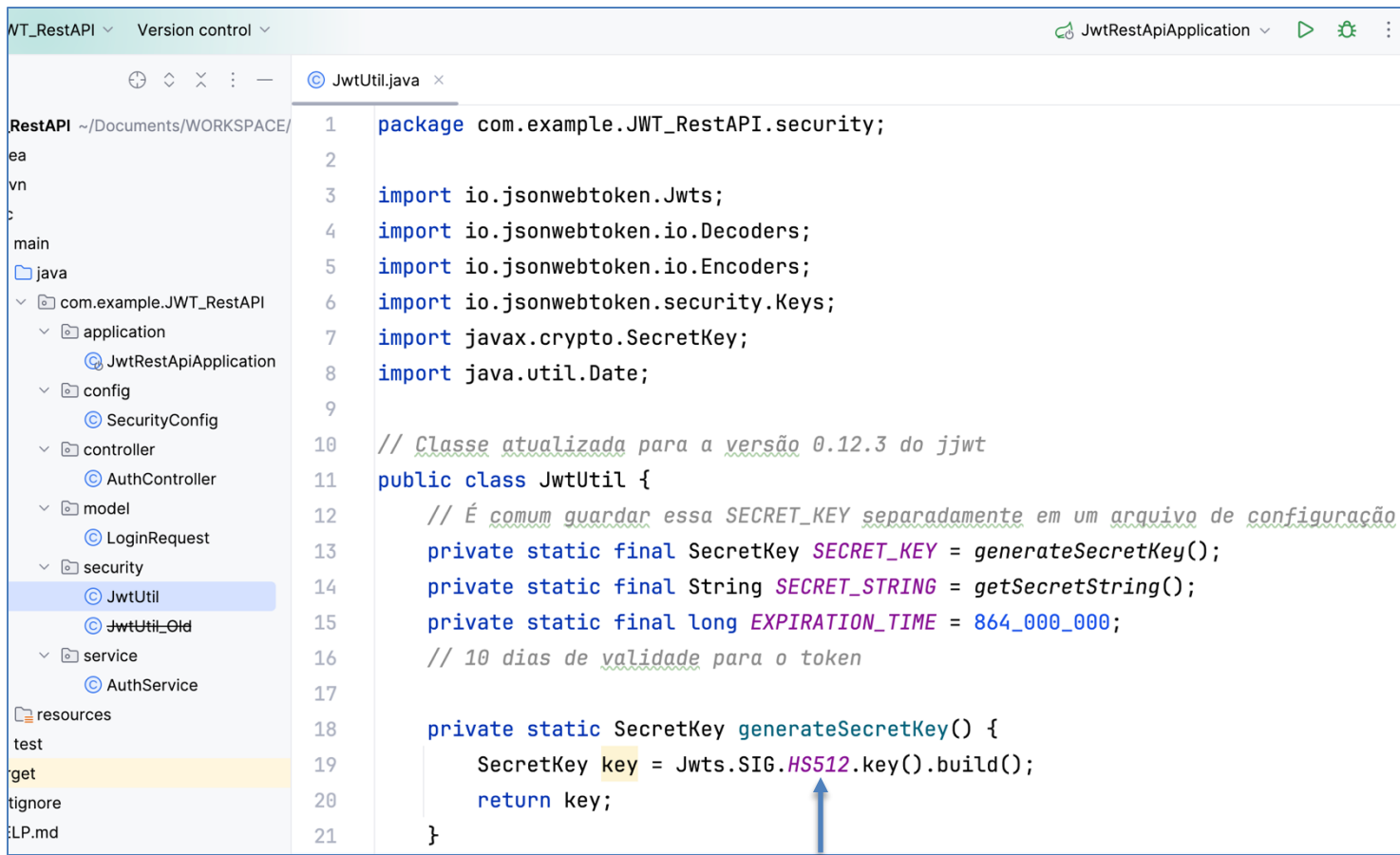
Foi atualizada para utilizar os métodos da versão 0.12.3 do jjwt.

The screenshot shows an IDE window with the following components:

- Top Bar:** Displays the project name "JWT_RestAPI" and the "Version control" status.
- Left Panel (Project Explorer):** Shows the project structure. The "security" package is expanded, and "JwtUtil" is selected. Other visible packages include "main", "application", "config", "controller", "model", "service", "resources", "test", and "ignore".
- Editor Window:** Displays the code for "JwtUtil.java". The code is as follows:

```
1 package com.example.JWT_RestAPI.security;
2
3 import io.jsonwebtoken.Jwts;
4 import io.jsonwebtoken.io.Decoders;
5 import io.jsonwebtoken.io.Encoders;
6 import io.jsonwebtoken.security.Keys;
7 import javax.crypto.SecretKey;
8 import java.util.Date;
9
10 // Classe atualizada para a versão 0.12.3 do jjwt
11 public class JwtUtil {
12     // É comum guardar essa SECRET_KEY separadamente em um arquivo de configuração
13     private static final SecretKey SECRET_KEY = generateSecretKey();
14     private static final String SECRET_STRING = getSecretString();
15     private static final long EXPIRATION_TIME = 864_000_000;
16     // 10 dias de validade para o token
17
18     private static SecretKey generateSecretKey() {
19         SecretKey key = Jwts.SIG.HS256.key().build();
20         return key;
21     }
```

Aqui estamos utilizando
o algoritmo criptográfico
HS512 (HMAC com SHA-
512):



```
1 package com.example.JWT_RestAPI.security;
2
3 import io.jsonwebtoken.Jwts;
4 import io.jsonwebtoken.io.Decoders;
5 import io.jsonwebtoken.io.Encoders;
6 import io.jsonwebtoken.security.Keys;
7 import javax.crypto.SecretKey;
8 import java.util.Date;
9
10 // Classe atualizada para a versão 0.12.3 do jjwt
11 public class JwtUtil {
12     // É comum guardar essa SECRET_KEY separadamente em um arquivo de configuração
13     private static final SecretKey SECRET_KEY = generateSecretKey();
14     private static final String SECRET_STRING = getSecretString();
15     private static final long EXPIRATION_TIME = 864_000_000;
16     // 10 dias de validade para o token
17
18     private static SecretKey generateSecretKey() {
19         SecretKey key = Jwts.SIG.HS512.key().build();
20         return key;
21     }
```

Métodos:

- generateSecretKey()
- getSecretString()
- generateToken(username)
- extractUsername(token)

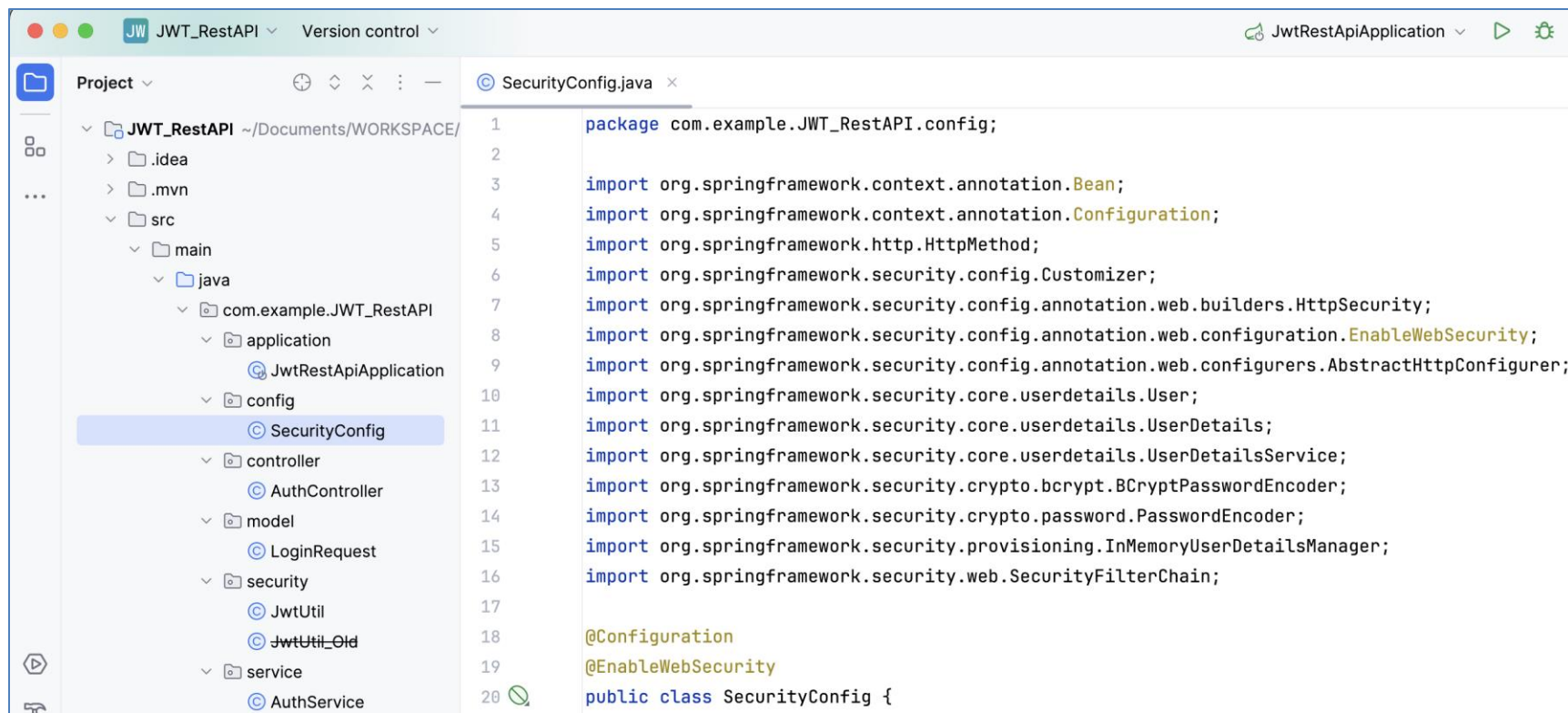
Pacote:

- security



```
23 private static String getSecretString() {
24     String secretString = Encoders.BASE64.encode(SECRET_KEY.getEncoded());
25     System.out.println("Secret Key: " + secretString);
26     return secretString;
27 }
28 public static String generateToken(String username) {
29     String token = Jwts.builder()
30         .subject(username)
31         .expiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
32         .signWith(SECRET_KEY)
33         .compact();
34     System.out.println("Token: " + token);
35     return token;
36 }
37 public static String extractUsername(String token) {
38     SecretKey secret = Keys.hmacShaKeyFor(Decoders.BASE64.decode(SECRET_STRING));
39     return Jwts.parser().verifyWith(secret).build().parseSignedClaims(token).
40         getPayload().getSubject();
41 }
42 }
```

- Classe **SecurityConfig** com as configurações de segurança.
 - Pacote config.



The screenshot shows an IDE window with the title bar "JWT_RestAPI" and "Version control". The left sidebar displays a project tree for "JWT_RestAPI" located at "~/Documents/WORKSPACE/". The tree structure includes folders for ".idea", ".mvn", "src", "main", "java", "com.example.JWT_RestAPI", "application", "config", "controller", "model", "security", "service", and "AuthService". The "config" folder is expanded, and "SecurityConfig" is selected. The main editor area shows the code for "SecurityConfig.java".

```
1 package com.example.JWT_RestAPI.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.http.HttpMethod;
6 import org.springframework.security.config.Customizer;
7 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
8 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
9 import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
10 import org.springframework.security.core.userdetails.User;
11 import org.springframework.security.core.userdetails.UserDetails;
12 import org.springframework.security.core.userdetails.UserDetailsService;
13 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
14 import org.springframework.security.crypto.password.PasswordEncoder;
15 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
16 import org.springframework.security.web.SecurityFilterChain;
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
```

- **@Configuration** Annotation: Indica ao Spring que essa classe é de configuração.
- Em outras palavras, ela será usada para definir configurações relacionadas à aplicação.



```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

- **@EnableWebSecurity** Annotation: Esta anotação é uma indicação para o Spring de que você deseja habilitar a configuração de segurança para a aplicação. Ela importa automaticamente as configurações padrão do Spring Security.

The screenshot shows an IDE window for a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with the 'SecurityConfig' class highlighted under the 'config' directory. The main editor area shows the code for 'SecurityConfig.java'. A blue arrow points to the '@EnableWebSecurity' annotation on line 19. The code includes imports for '@Configuration' and '@Bean', and a 'public class SecurityConfig' containing a '@Bean' annotated method 'securityFilterChain' that configures HTTP security with various request matchers and roles.

```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

- **@Bean** Annotation: O método **securityFilterChain** é anotado com **@Bean** e retorna uma instância de **SecurityFilterChain**. Esta instância define as configurações específicas para a segurança da aplicação.

The screenshot shows an IDE window for a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with 'SecurityConfig' selected under the 'config' directory. The main editor area shows the 'SecurityConfig.java' file. The code defines a Spring configuration class with the following annotations and methods:

```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

A blue arrow points to the `@Bean` annotation on line 22, highlighting its role in registering the `securityFilterChain` method as a Spring bean.

- O método **securityFilterChain** está configurando o filtro de segurança para o Spring Security. Ele definirá a cadeia de filtros de segurança que serão aplicados às requisições HTTP.

The screenshot shows an IDE window for a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with 'SecurityConfig' selected under the 'config' directory. The main editor area shows the code for 'SecurityConfig.java'. The code includes annotations for configuration and web security, and a bean definition for the security filter chain. A blue arrow points from the text above to the 'securityFilterChain' method in the code.

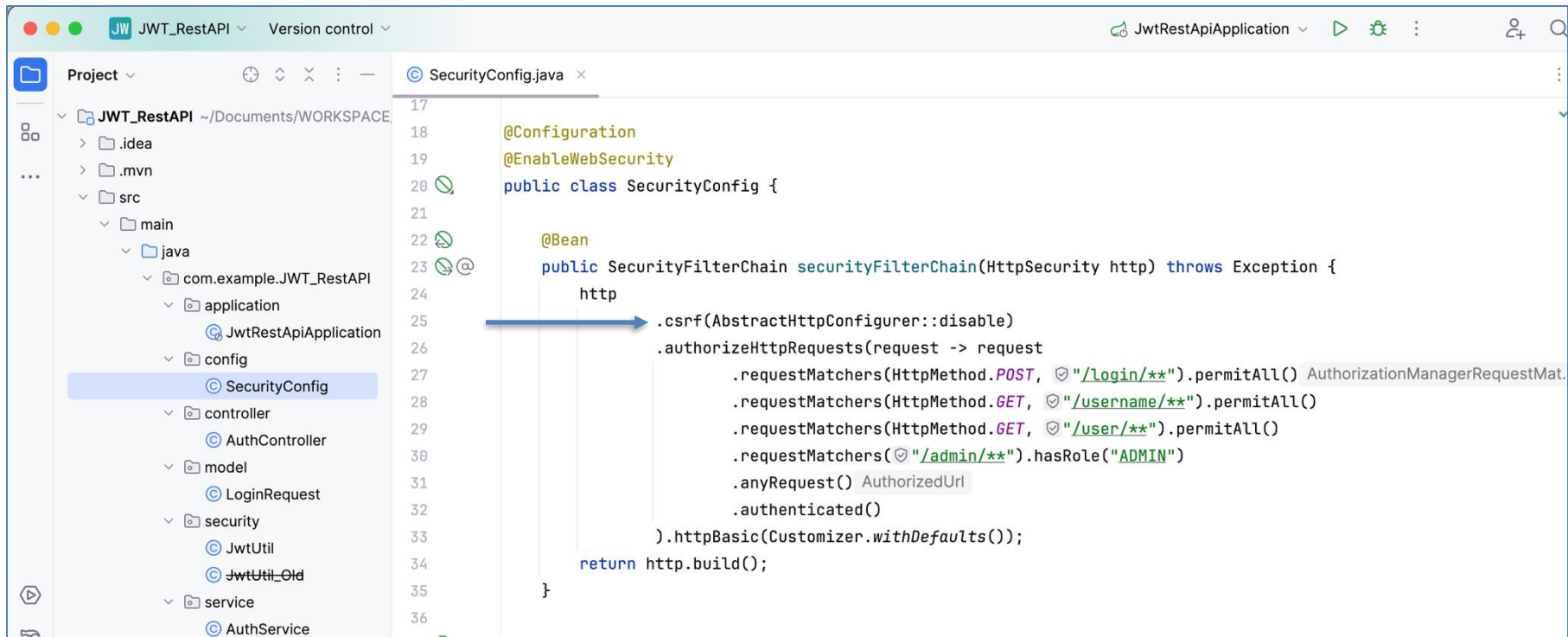
```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

- O parâmetro **HttpSecurity** representa a configuração do Spring Security para a aplicação web. Ele é passado para o método pelo próprio Spring. Vejamos a seguir sua configuração.

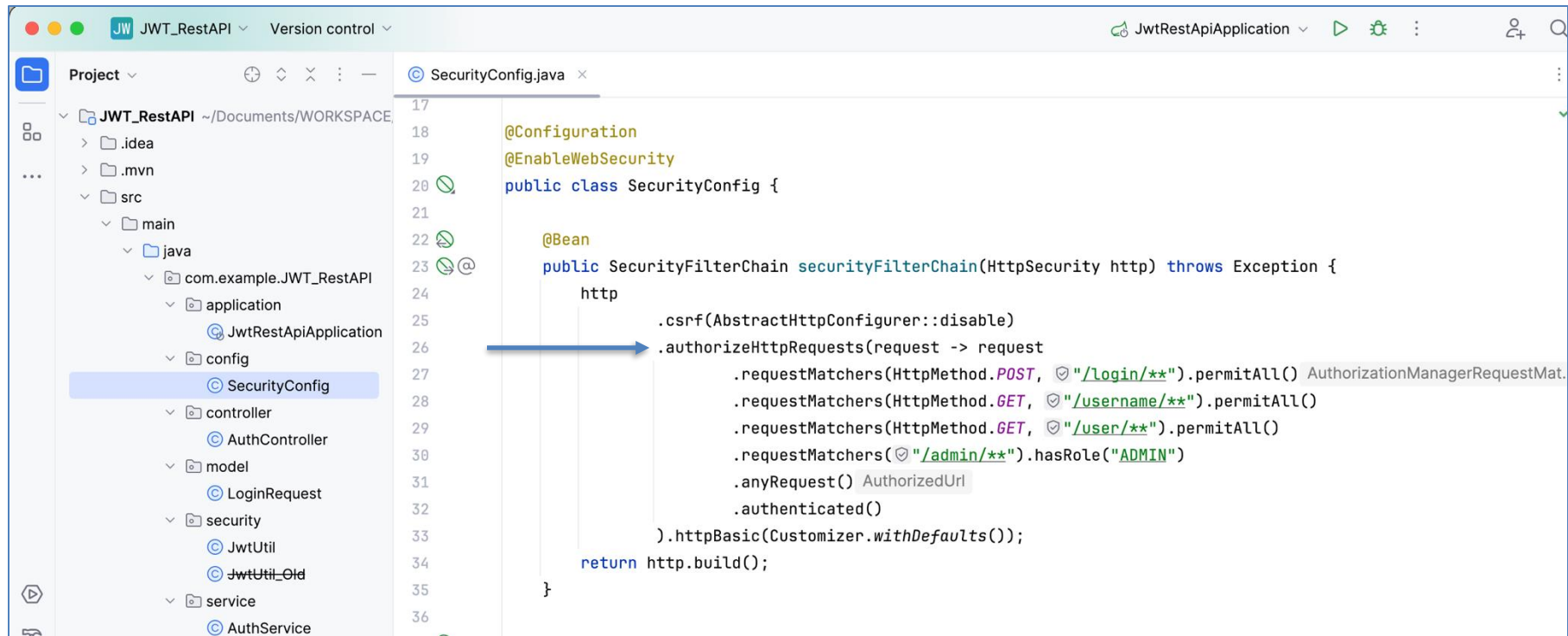


```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

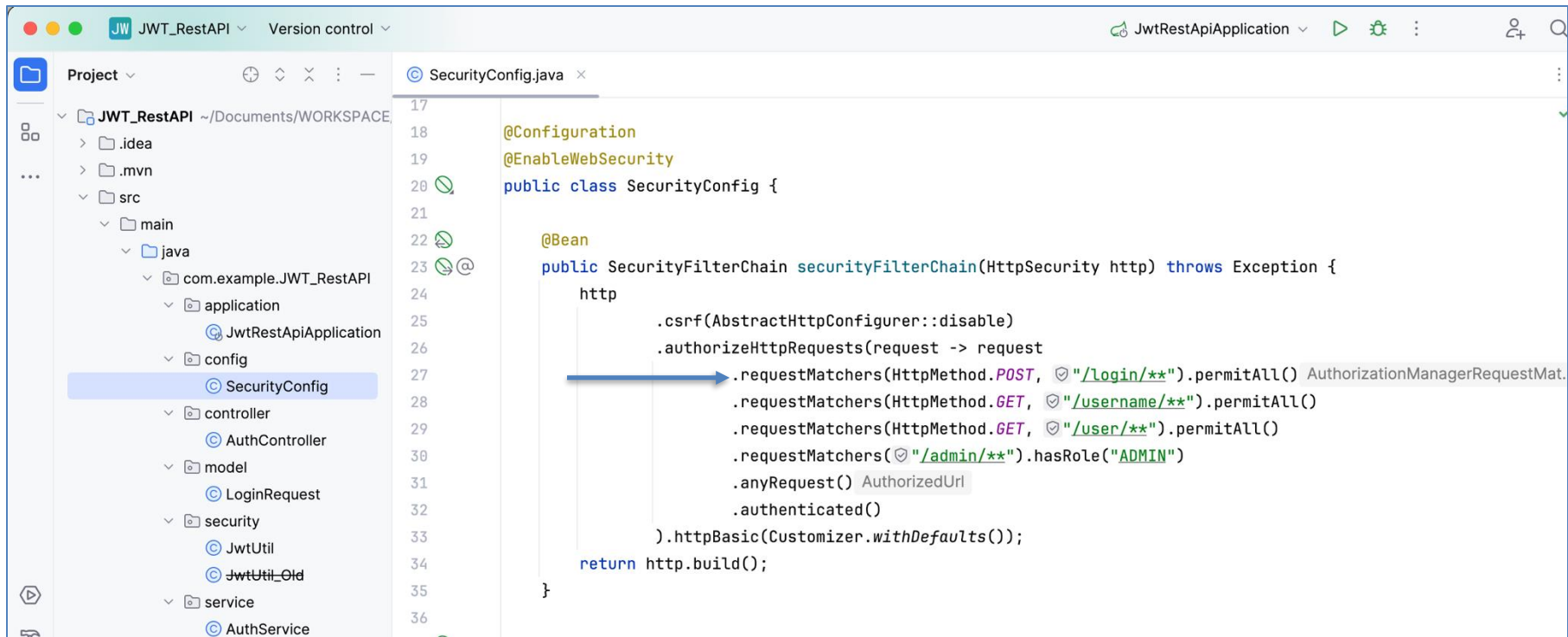
- **.csrf(AbstractHttpConfigurer::disable):** Desativa a proteção contra CSRF (Cross-Site Request Forgery). Em muitos casos, desabilitar o CSRF é apropriado para APIs RESTful, mas em aplicações web tradicionais, isso deve ser feito com cautela.



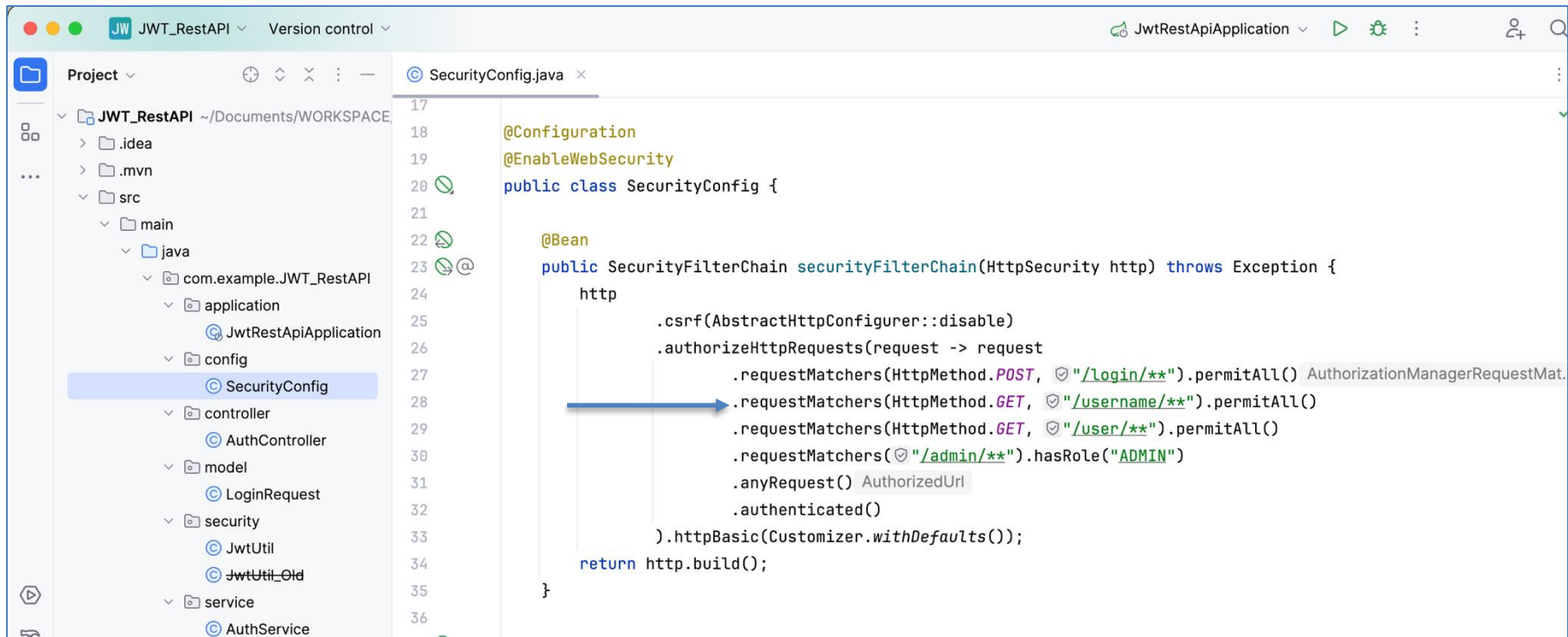
- `.authorizeHttpRequests(request -> {...})`:
- Configura as regras de autorização para diferentes tipos de requisições HTTP.



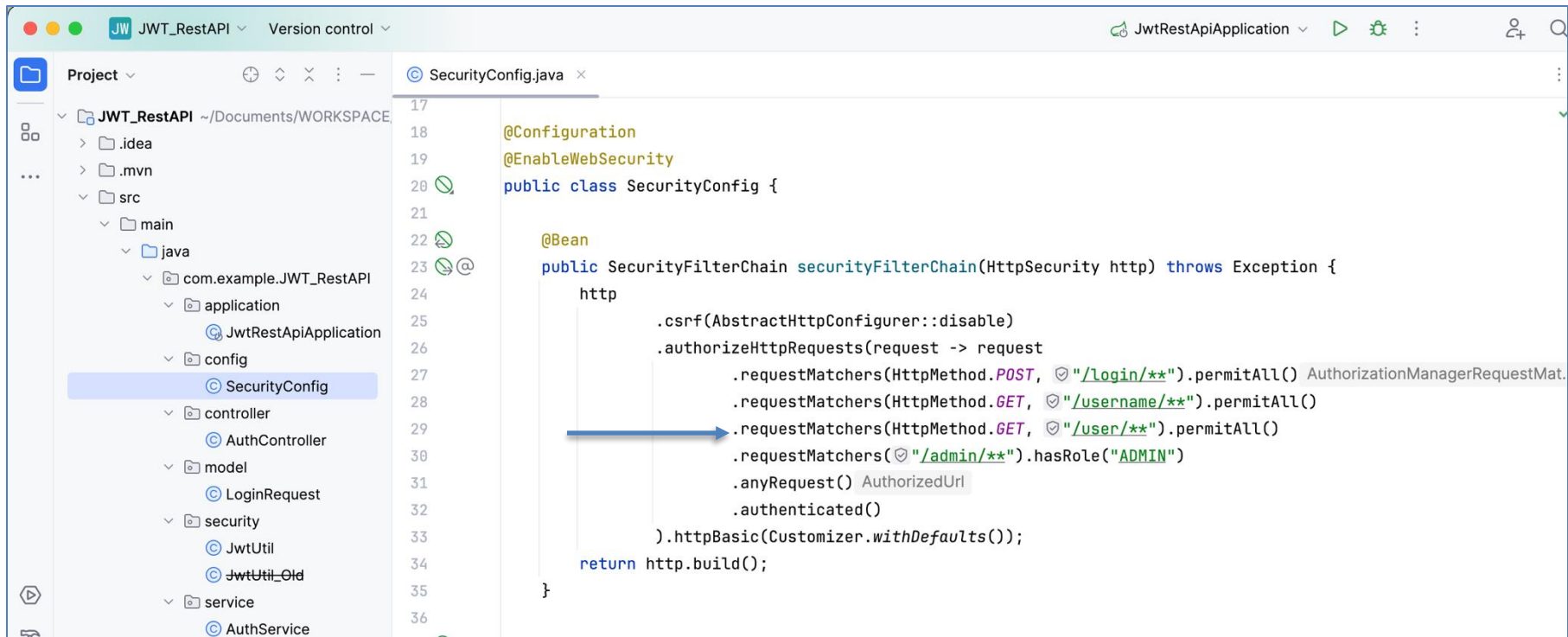
- `.requestMatchers(HttpMethod.POST, "/login/**").permitAll();`
- Permite requisições POST para URLs que começam com `"/login/"` sem autenticação.



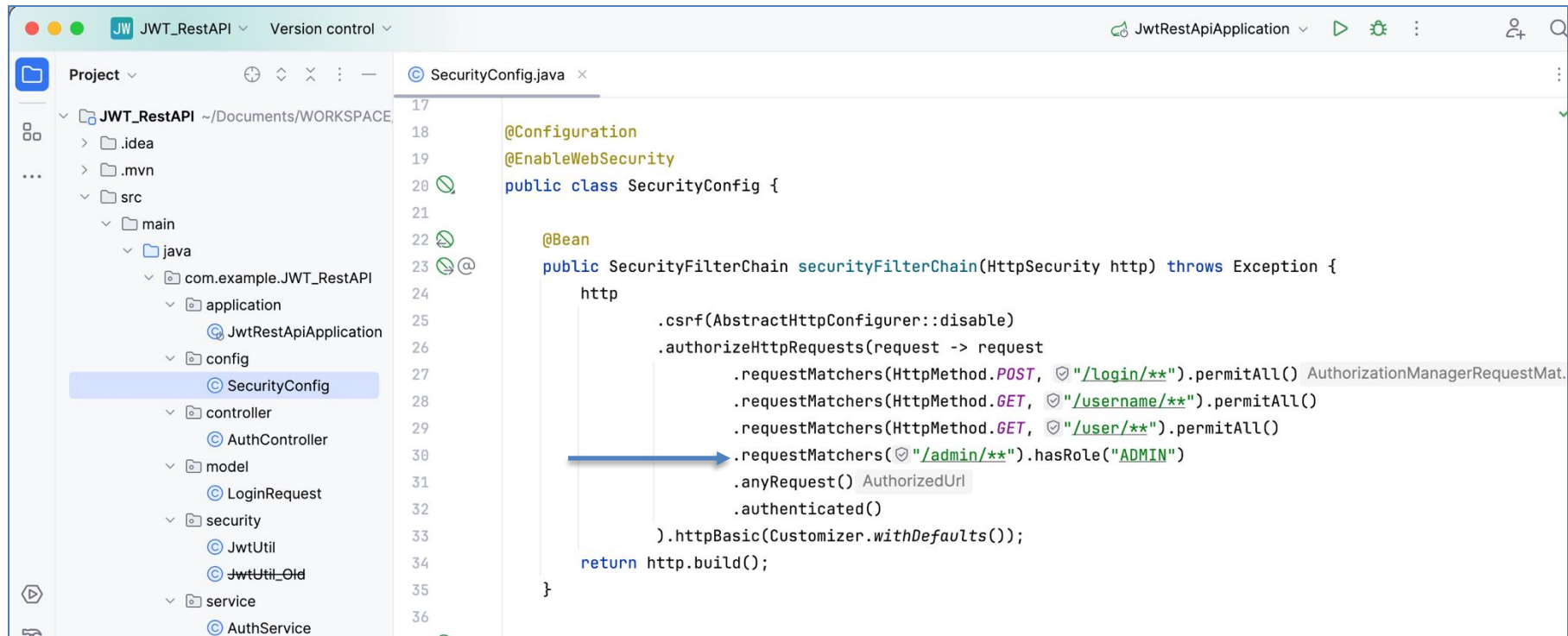
- `.requestMatchers(HttpMethod.GET, "/username/**").permitAll():`
- Permite requisições GET para URLs que começam com `"/username/"` sem autenticação.



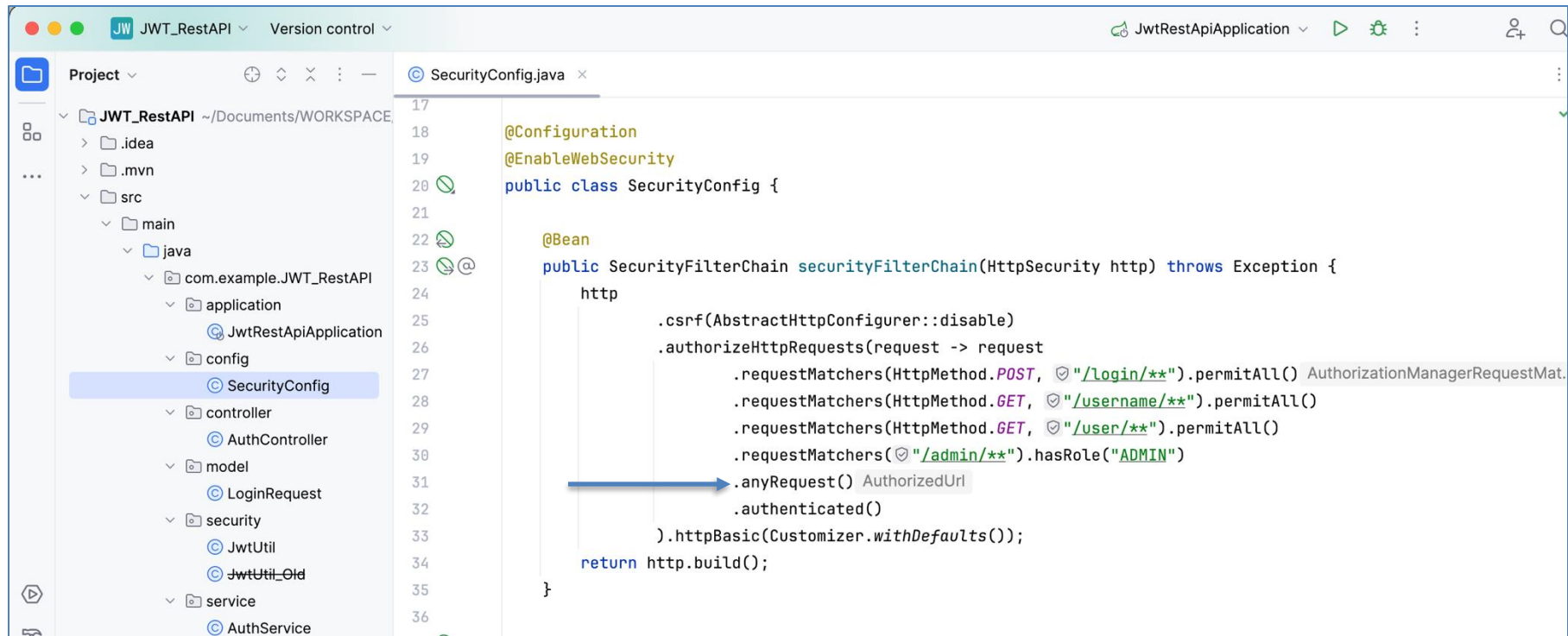
- `.requestMatchers(HttpMethod.GET, "/user/**").permitAll();`
- Permite requisições GET para URLs que começam com `"/user/"` sem autenticação.



- `.requestMatchers("/admin/**").hasRole("ADMIN")`:
- Exige que as requisições para URLs que começam com `"/admin/"` tenham o papel (role) `"ADMIN"`.



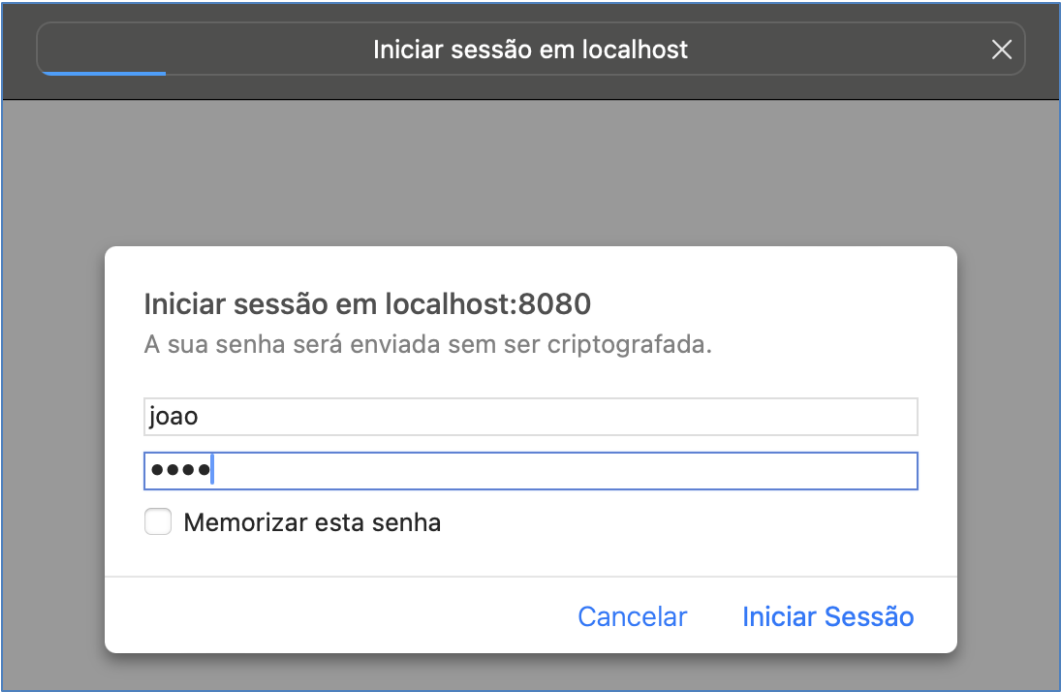
- `.anyRequest().authenticated()`:
- Todas as outras requisições devem ser autenticadas.



- **.httpBasic(Customizer.withDefaults())**: Configura a autenticação HTTP básica. Isso significa que, ao acessar áreas protegidas, o navegador exibirá uma caixa de diálogo para inserir credenciais de usuário.

```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

- **.httpBasic(Customizer.withDefaults())**: Configura a autenticação HTTP básica. Isso significa que, ao acessar áreas protegidas, o navegador exibirá uma caixa de diálogo para inserir credenciais de usuário.



- **return http.build();**: Retorna o objeto `HttpSecurity` configurado como um `SecurityFilterChain`. Isso completa a configuração da cadeia de filtros de segurança.

```
17
18 @Configuration
19 @EnableWebSecurity
20 public class SecurityConfig {
21
22     @Bean
23     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(AbstractHttpConfigurer::disable)
26             .authorizeHttpRequests(request -> request
27                 .requestMatchers(HttpMethod.POST, "/login/**").permitAll() AuthorizationManagerRequestMat.
28                 .requestMatchers(HttpMethod.GET, "/username/**").permitAll()
29                 .requestMatchers(HttpMethod.GET, "/user/**").permitAll()
30                 .requestMatchers("/admin/**").hasRole("ADMIN")
31                 .anyRequest() AuthorizedUrl
32                 .authenticated()
33             ).httpBasic(Customizer.withDefaults());
34         return http.build();
35     }
36 }
```

- **userDetailsService()** para configurar os usuários:
- Este método cria um UserDetailsService, que é uma interface do Spring Security responsável por recuperar informações do usuário durante o processo de autenticação.



```
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
56  
@Bean  
public UserDetailsService userDetailsService() {  
    UserDetails user = User.builder()  
        .username("joao")  
        .password(passwordEncoder().encode(rawPassword: "4321"))  
        .roles("USER")  
        .build();  
    UserDetails admin = User.builder()  
        .username("admin")  
        .password(passwordEncoder().encode(rawPassword: "1234"))  
        .roles("ADMIN")  
        .build();  
    return new InMemoryUserDetailsManager(user, admin);  
}  
  
@Bean  
> public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }  
}
```

- **userDetailsService()** para configurar os usuários:
- Para cada usuário, são configurados detalhes como nome de usuário, senha (já codificada usando o codificador de senha), e as roles associadas ao usuário.



```
37 @Bean
38 public UserDetailsService userDetailsService() {
39     UserDetails user = User.builder()
40         .username("joao")
41         .password(passwordEncoder().encode(rawPassword: "4321"))
42         .roles("USER")
43         .build();
44     UserDetails admin = User.builder()
45         .username("admin")
46         .password(passwordEncoder().encode(rawPassword: "1234"))
47         .roles("ADMIN")
48         .build();
49     return new InMemoryUserDetailsManager(user, admin);
50 }
51
52 @Bean
53 public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
56 }
```

- **userDetailsService()** para configurar os usuários:
- Ambos os usuários são armazenados em memória (usando **InMemoryUserDetailsManager**), o que é útil para cenários de desenvolvimento e testes.



The screenshot shows an IDE window for a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with 'SecurityConfig' selected under the 'config' directory. The main editor area shows the code for 'SecurityConfig.java'. A blue arrow points to the 'userDetailsService()' method, which is annotated with '@Bean'. This method creates two users: 'joao' (USER) and 'admin' (ADMIN), and returns an 'InMemoryUserDetailsManager' instance. The 'passwordEncoder()' method is also shown, returning a 'BCryptPasswordEncoder' instance.

```
37 @Bean
38 public UserDetailsService userDetailsService() {
39     UserDetails user = User.builder()
40         .username("joao")
41         .password(passwordEncoder().encode(rawPassword: "4321"))
42         .roles("USER")
43         .build();
44     UserDetails admin = User.builder()
45         .username("admin")
46         .password(passwordEncoder().encode(rawPassword: "1234"))
47         .roles("ADMIN")
48         .build();
49     return new InMemoryUserDetailsManager(user, admin);
50 }
51
52 @Bean
53 public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
56 }
```

- **userDetailsService()** para configurar os usuários:
- Está anotado com o **@Bean**, indicando que o objeto que ele retorna será gerenciado pelo contêiner Spring e pode ser injetado em outros componentes ou configurações da aplicação.



```
37
38
39 public UserDetailsService userDetailsService() {
40     UserDetails user = User.builder()
41         .username("joao")
42         .password(passwordEncoder().encode(rawPassword: "4321"))
43         .roles("USER")
44         .build();
45     UserDetails admin = User.builder()
46         .username("admin")
47         .password(passwordEncoder().encode(rawPassword: "1234"))
48         .roles("ADMIN")
49         .build();
50     return new InMemoryUserDetailsManager(user, admin);
51 }
52
53 @Bean
54 public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
55
56 }
```

- **passwordEncoder()** para configurar o codificador de senha:
- O método `passwordEncoder()` cria um `PasswordEncoder`. Neste caso, está usando o `BCryptPasswordEncoder`, que é um algoritmo de hash de senha forte e seguro.



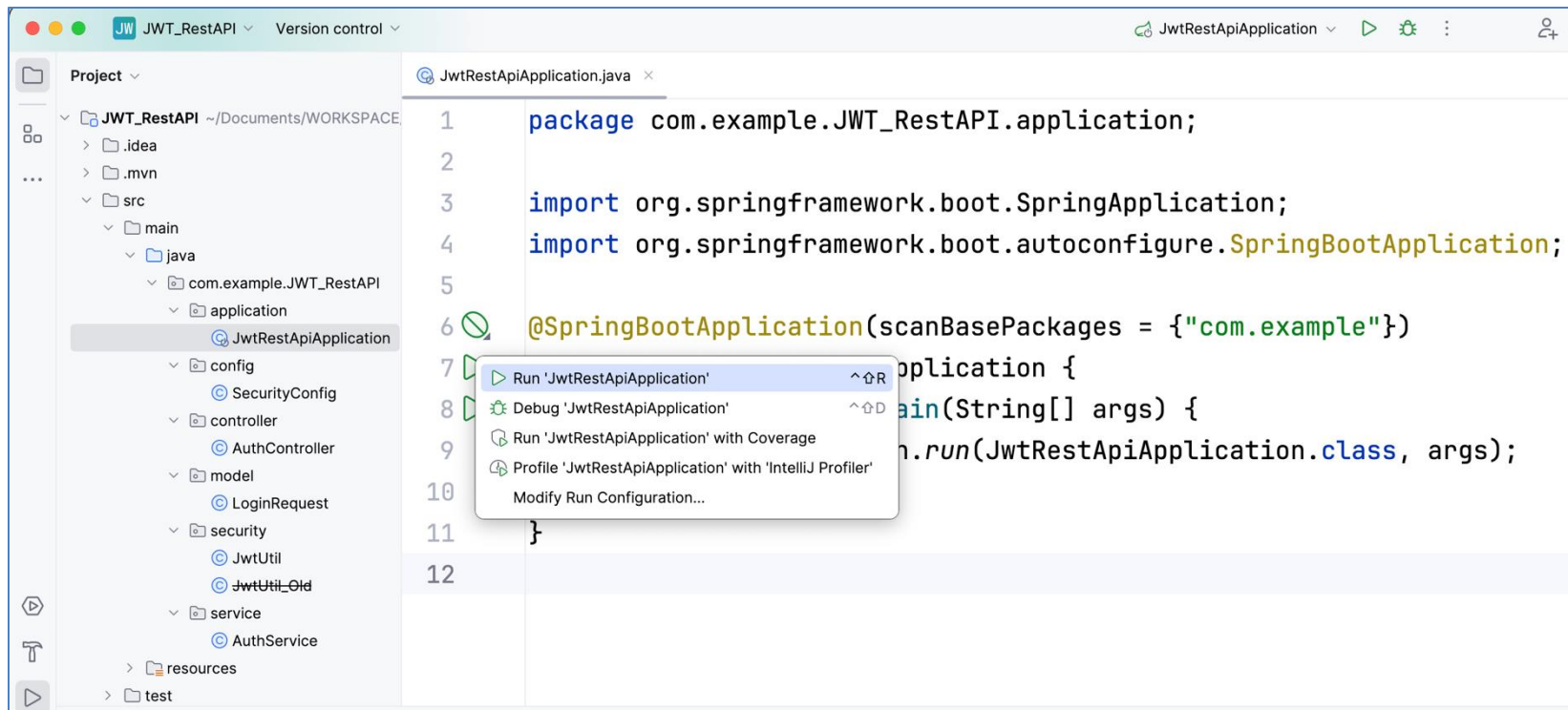
The screenshot shows an IDE window for a project named 'JWT_RestAPI'. The left sidebar displays the project structure, with the 'SecurityConfig' class selected under the 'config' package. The main editor area shows the code for 'SecurityConfig.java'. The code defines a `@Bean` method `userDetailsService()` that creates two users: 'joao' with password '4321' and role 'USER', and 'admin' with password '1234' and role 'ADMIN'. A blue arrow points from the `passwordEncoder()` call in the 'admin' user creation to the `passwordEncoder()` method definition below, which returns a new `BCryptPasswordEncoder()`.

```
37 @Bean
38 public UserDetailsService userDetailsService() {
39     UserDetails user = User.builder()
40         .username("joao")
41         .password(passwordEncoder().encode(rawPassword: "4321"))
42         .roles("USER")
43         .build();
44     UserDetails admin = User.builder()
45         .username("admin")
46         .password(passwordEncoder().encode(rawPassword: "1234"))
47         .roles("ADMIN")
48         .build();
49     return new InMemoryUserDetailsManager(user, admin);
50 }
51
52 @Bean
53 public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
56 }
```

- O **BCryptPasswordEncoder** é comumente usado para armazenar senhas de forma segura, aplicando um algoritmo de hash irreversível com um fator de custo ajustável, tornando as senhas mais difíceis de serem comprometidas em caso de vazamento.

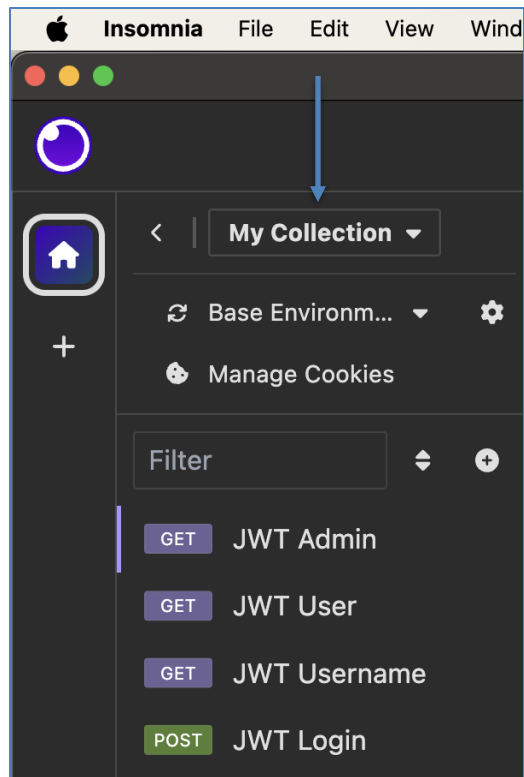
```
37 @Bean
38 public UserDetailsService userDetailsService() {
39     UserDetails user = User.builder()
40         .username("joao")
41         .password(passwordEncoder().encode( rawPassword: "4321"))
42         .roles("USER")
43         .build();
44     UserDetails admin = User.builder()
45         .username("admin")
46         .password(passwordEncoder().encode( rawPassword: "1234"))
47         .roles("ADMIN")
48         .build();
49     return new InMemoryUserDetailsManager(user,admin);
50 }
51
52 @Bean
53 public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
56 }
```

- Podemos agora testar as requisições utilizando o Postman ou o Insomnia.
- Primeiro iniciamos a aplicação:



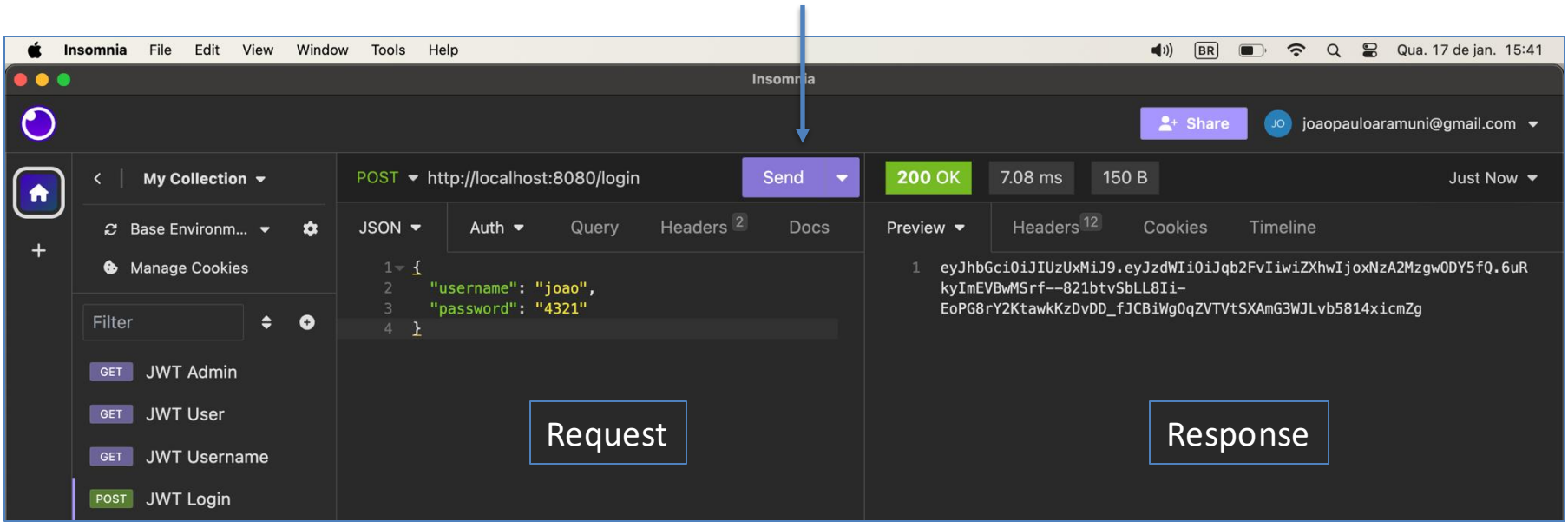
Testando as requisições

- Testaremos as seguintes URLs:
 - POST <http://localhost:8080/login>
 - `generateToken(String username)`
 - GET <http://localhost:8080/username/{token}>
 - `extractUsername(String token)`
 - GET <http://localhost:8080/user>
 - `getUser(Authentication authentication)`
 - GET <http://localhost:8080/admin>
 - `onlyAdmin(Authentication authentication)`
- Dica:
 - Salve e organize suas requisições no Postman ou Insomnia.



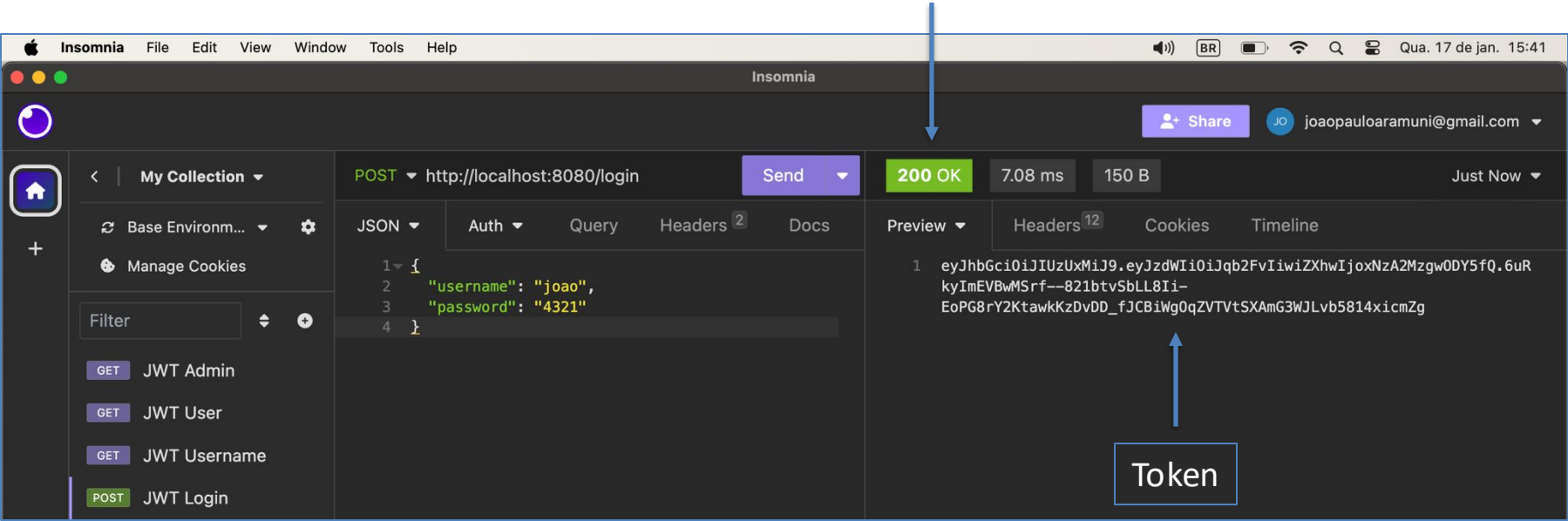
- POST <http://localhost:8080/login>
- @RequestBody: [JSON]

```
{  
    "username": "joao",  
    "password": "4321"  
}
```

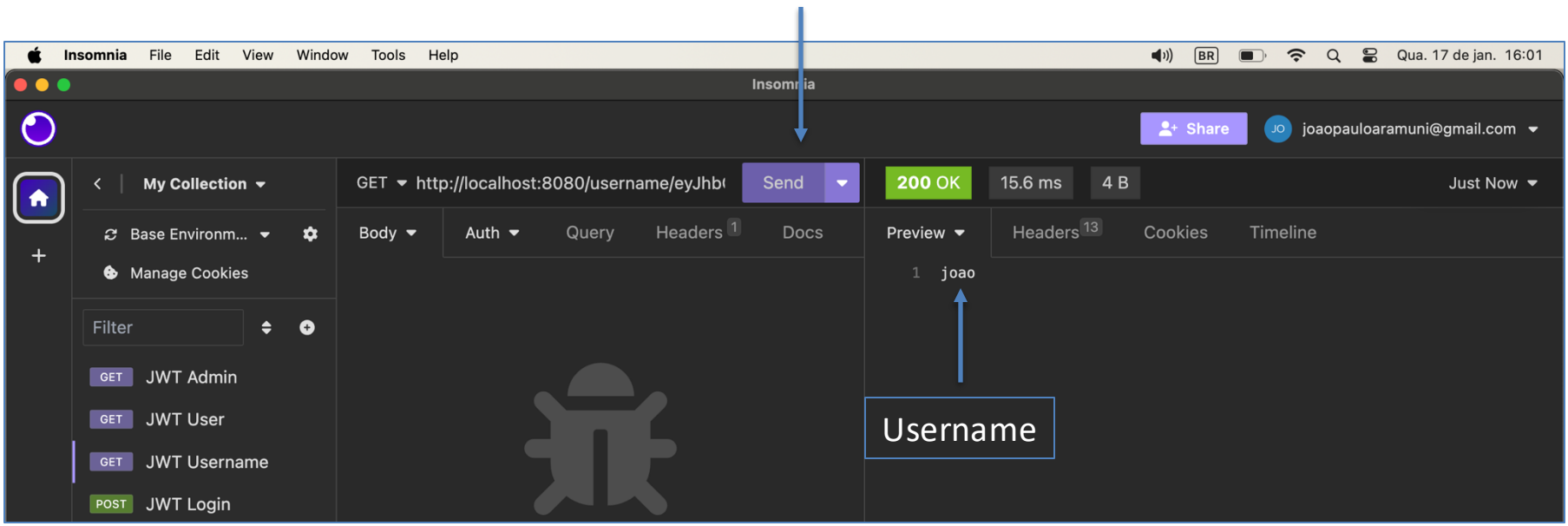


- POST <http://localhost:8080/login>
- @RequestBody: [JSON]

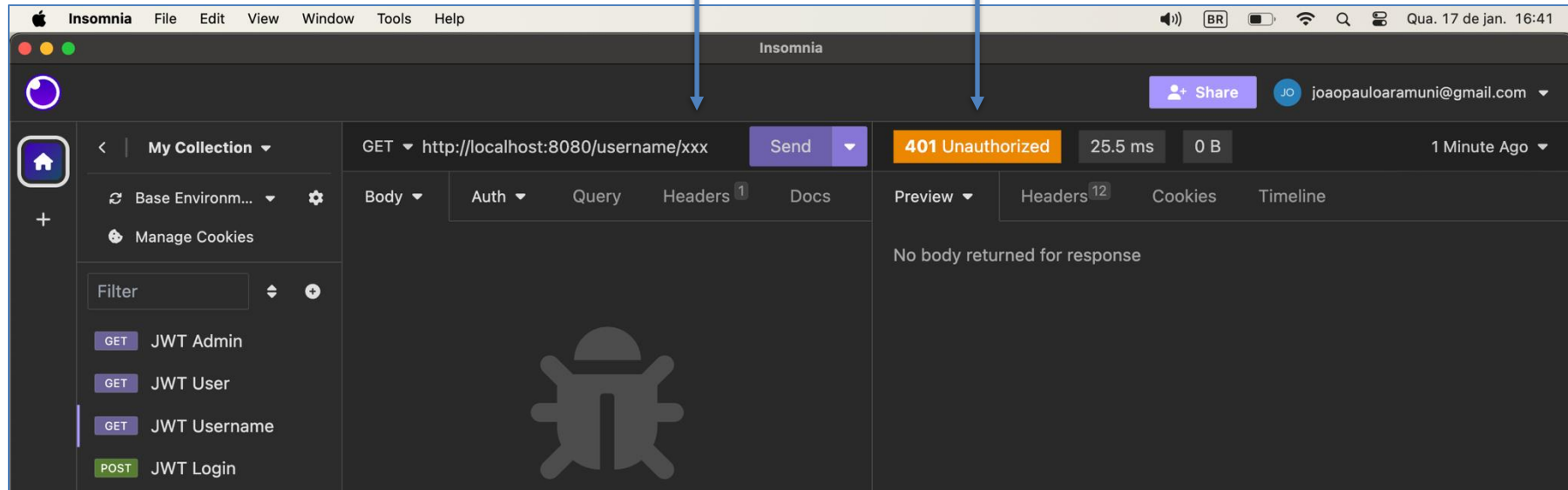
```
{  
    "username": "joao",  
    "password": "4321"  
}
```



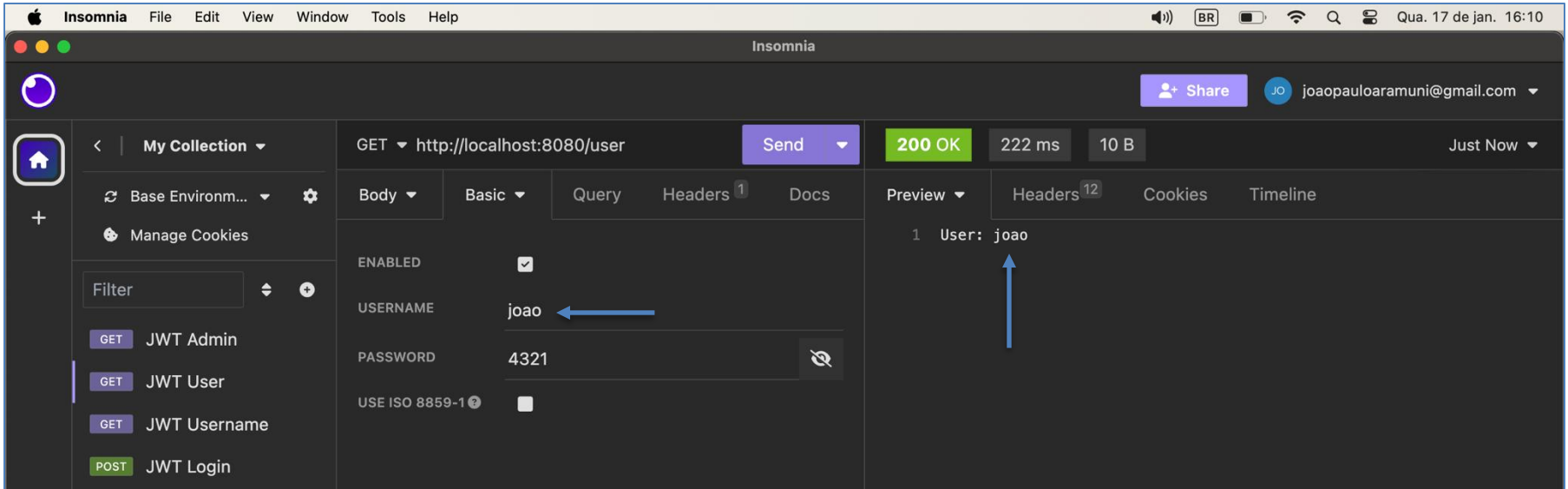
- GET <http://localhost:8080/username/{token}>
- Copie o token gerado e cole na URL:
- http://localhost:8080/username/eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJqb2FvliwiZXhwIjoxNzA2MzgwODY5fQ.6uRkyImEVBwMSrf--821btvSbLL8li-EoPG8rY2KtawkKzDvDD_fjCBiWgOqZTVtSXAmG3WJLvb5814xicmZg
- Veja o retorno do método extractUsername(String token):



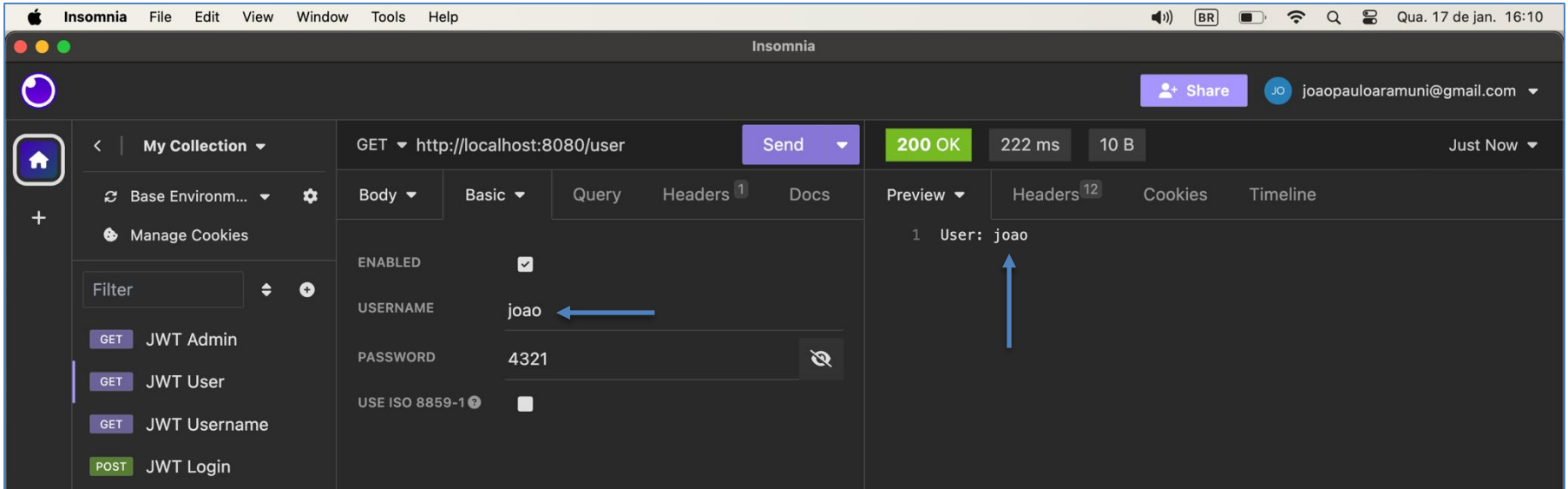
- GET <http://localhost:8080/username/{token}>
- Caso o token seja inválido, o seguinte erro **401 Unauthorized** irá aparecer:
- Somente tokens válidos, e dentro do período de validade, serão autorizados.



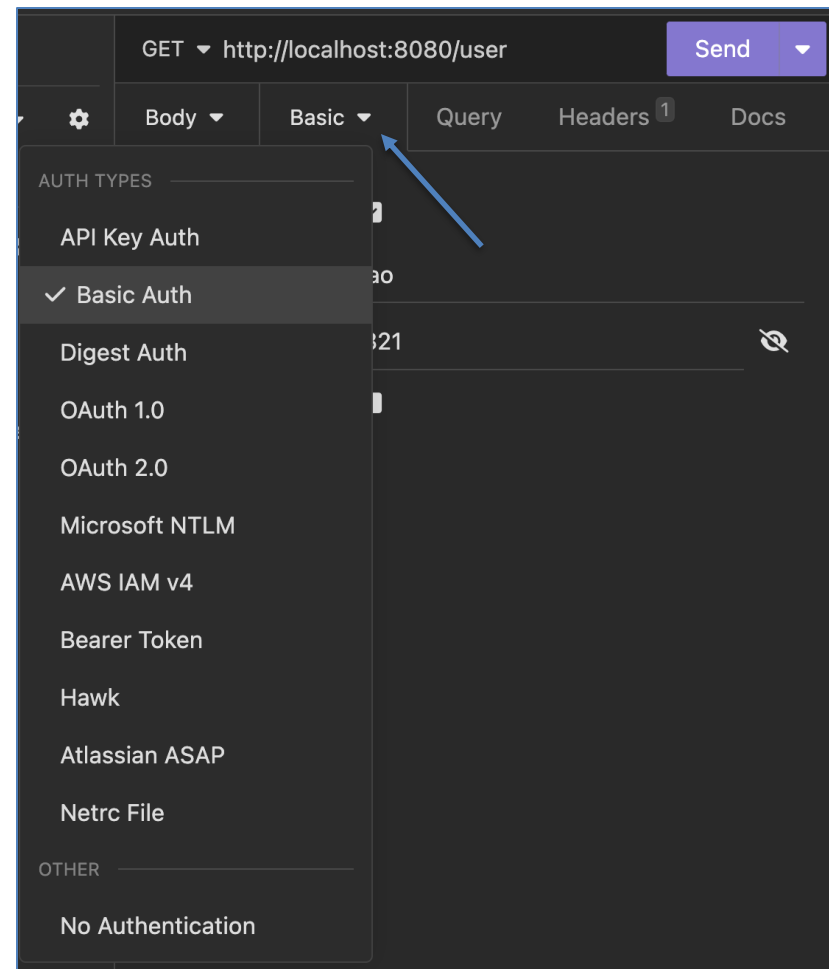
- GET <http://localhost:8080/user>
- Quando fazemos uma requisição autenticada para a aplicação, o Spring Security gera um objeto **Authentication** e o disponibiliza para métodos que precisam acessar informações sobre o usuário autenticado.
- USERNAME: joao; PASSWORD: 4321



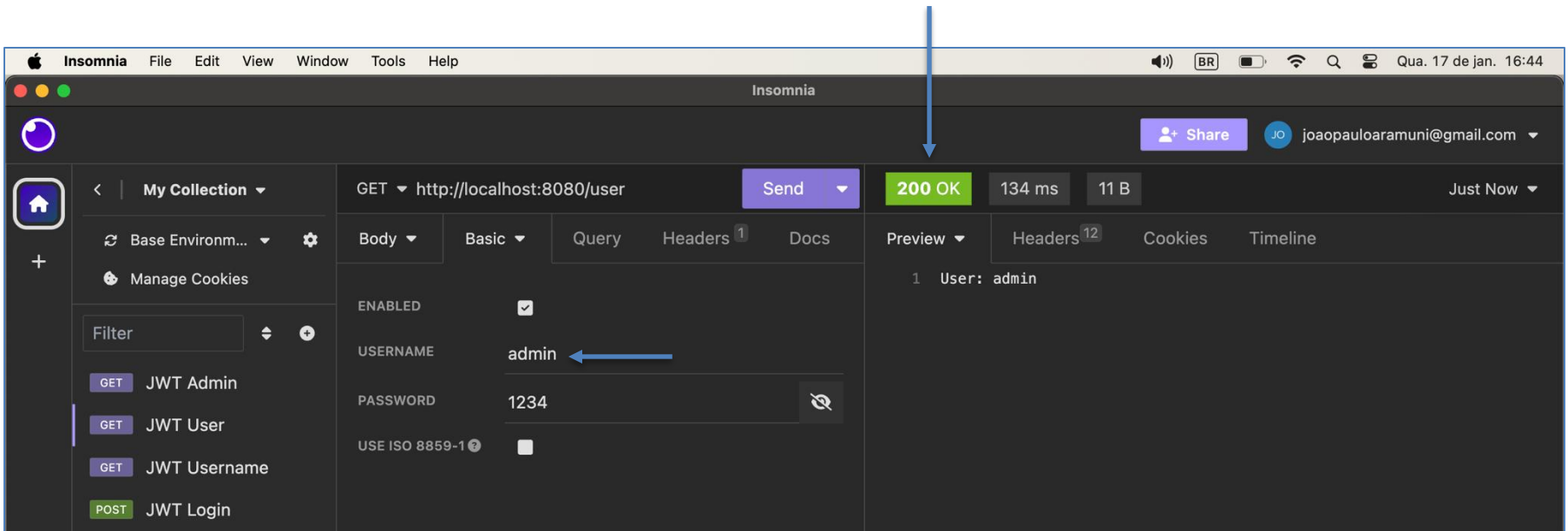
- GET <http://localhost:8080/user>
- Isso permite que você acesse e utilize essas informações em sua lógica de aplicação, como exibir detalhes do usuário em uma página da web ou realizar uma lógica específica com base nas permissões do usuário.
- `return "User: " + authentication.getName();`



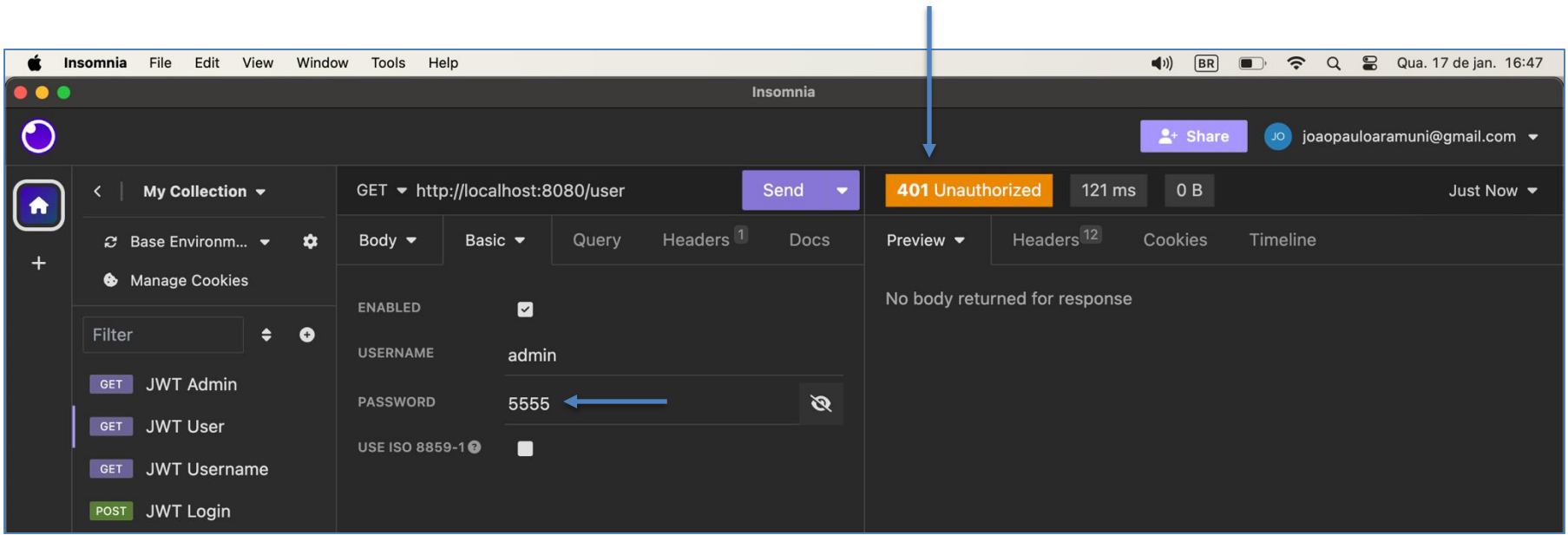
- GET <http://localhost:8080/user>
- Neste caso estamos utilizando **Basic Auth**.
- O **Basic Authentication** (Autenticação Básica) é um método simples de autenticação que é frequentemente utilizado em solicitações HTTP.
- Ele é especificado no cabeçalho da requisição e envolve a codificação de um nome de usuário e senha, que são combinados e enviados como uma string codificada em Base64.



- GET <http://localhost:8080/user>
- Para USERNAME admin; PASSWORD 1234 também funciona.
- Isso acontece por conta da configuração em **InMemoryUserDetailsManager**.
- Vejamos agora um caso de erro para /user.



- GET <http://localhost:8080/user>
- Vejamos agora alguns casos de erro para /user.
- USERNAME ou PASSWORD incorretos.
- **401 Unauthorized**

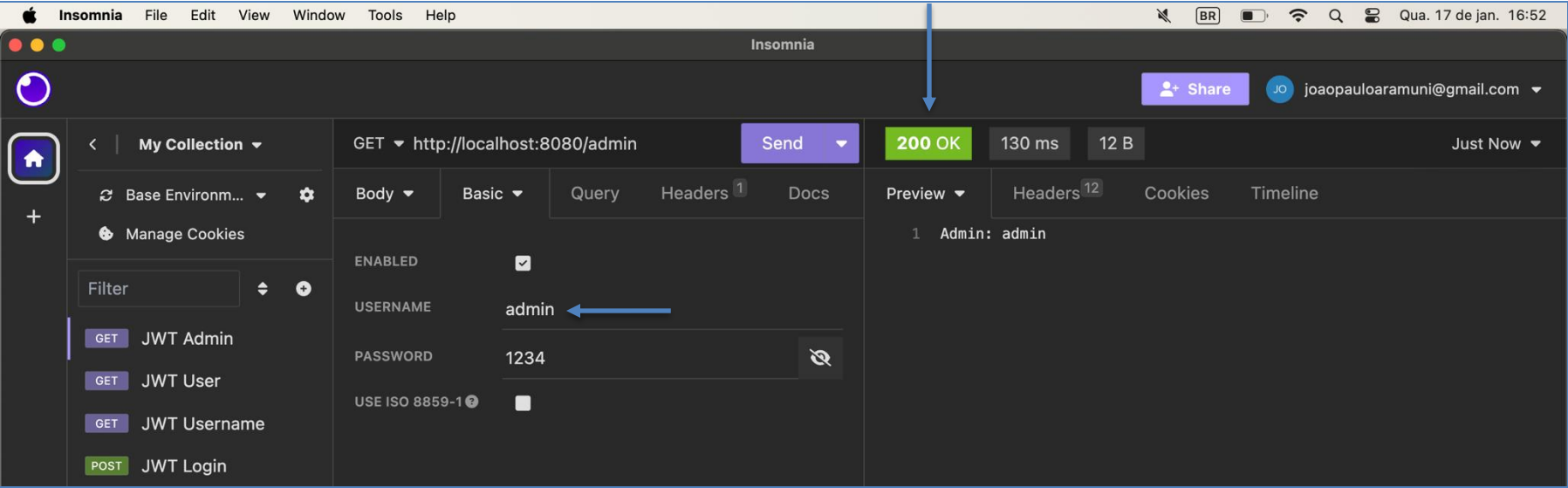


- GET <http://localhost:8080/admin>
- Por fim, /admin

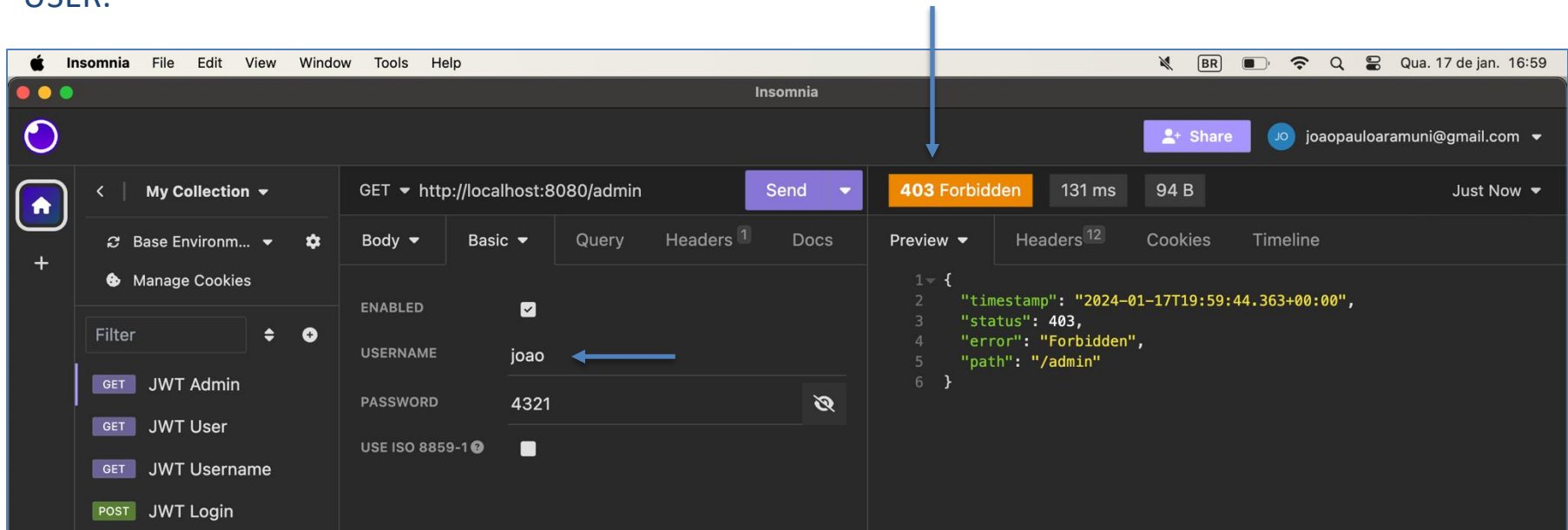
SecurityConfig

```
UserDetails admin = User.builder()  
    .username("admin")  
    .password(passwordEncoder().encode("1234"))  
    .roles("ADMIN")  
    .build();
```

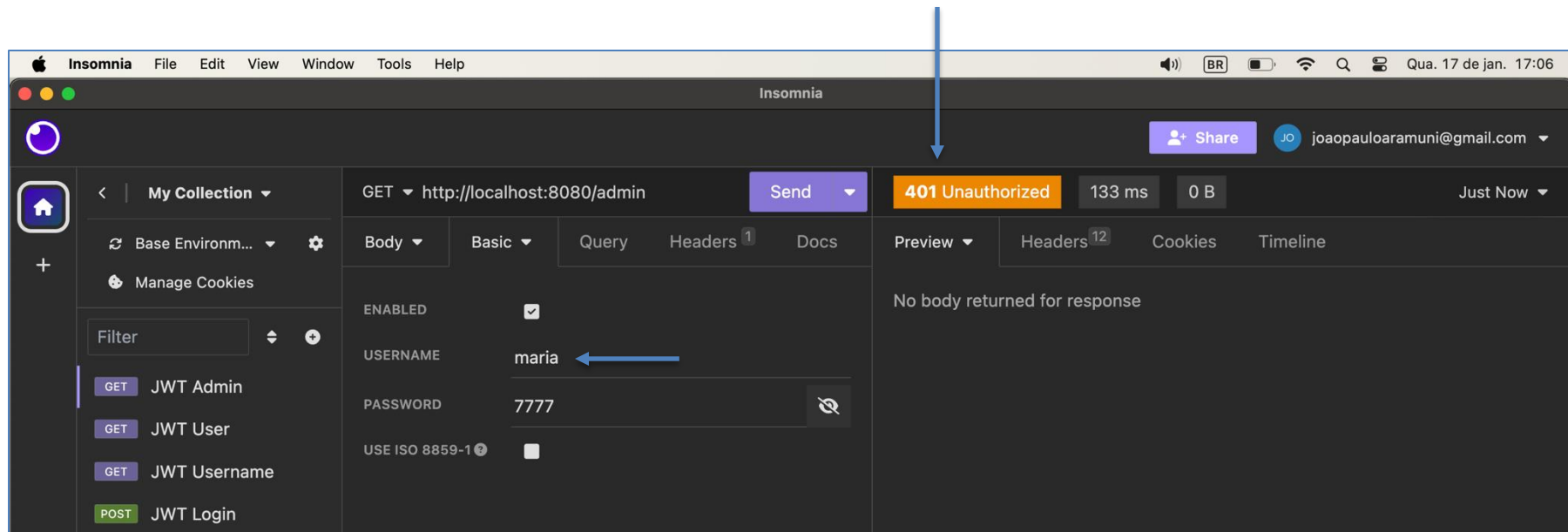
```
.authorizeHttpRequests(request -> request  
    .requestMatchers(HttpMethod.POST, "/login/**").permitAll()  
    .requestMatchers(HttpMethod.GET, "/username/**").permitAll()  
    .requestMatchers(HttpMethod.GET, "/user/**").permitAll()  
    .requestMatchers("/admin/**").hasRole("ADMIN")  
    .anyRequest().authenticated())
```



- GET <http://localhost:8080/admin>
- Veja que se tentamos uma requisição com um usuário que existe, o erro é:
- **403 Forbidden**
- Isso acontece porque o usuário joao não possui a role ADMIN. Somente a role USER.



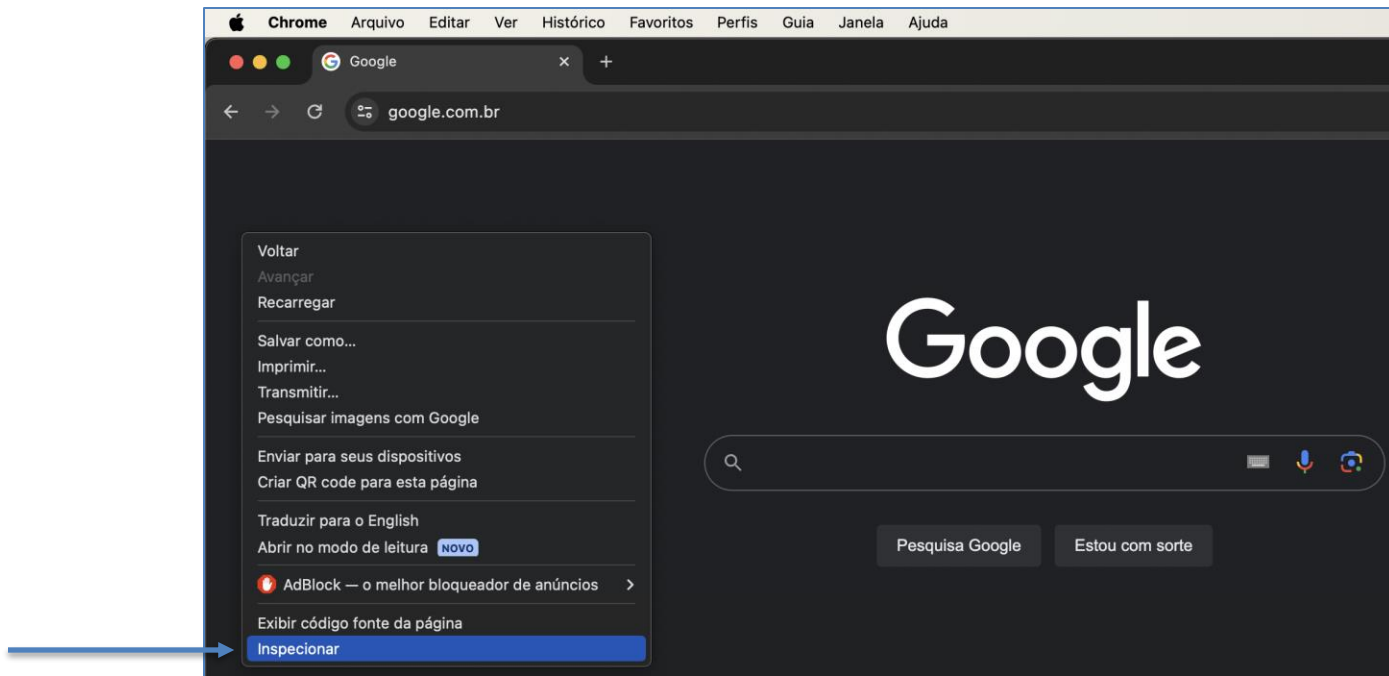
- GET <http://localhost:8080/admin>
- Se tentamos uma requisição com um usuário que **não** existe, o erro é:
- **401 Unauthorized**
- Com isso finalizamos os testes das nossas requisições HTTP.



Bônus



- Aba de **desenvolvedor** do Google Chrome (DevTools).
- Abra o seu Google Chrome, clique com o botão direito e vá até **Inspeccionar**.



Chrome Arquivo Editar Ver Histórico Favoritos Perfis Guia Janela Ajuda

Google

google.com.br

Gmail Imagens

img.InXdpd 272 x 92

Google

Pesquisa Google Estou com sorte

Brasil

Sobre Publicidade Negócios Como funciona a Pesquisa

Privacidade Termos Configurações

DevTools

Elements Console Sources Network Performance Memory Application

```
<!DOCTYPE html>
<html id=scope itentype="http://schema.org/WebPage" lang="pt-BR">
  <head>
    <body jsmodel="hspDDf" data-dt="1" jsaction="xjhtIf:.CLIENT;02V
yse:.CLIENT;IVKtfe:.CLIENT;Ez7Vmc:.CLIENT;YUC7He:.CLIENT;hWT9Jb:
.CLIENT;wCulWe:.CLIENT;VM8Bg:.CLIENT;qqf0n:.CLIENT;A8708b:.CLIE
T;Ycfj:.CLIENT;szj0R:.CLIENT;JL9Qdc:.CLIENT;KwLxhc:.CLIENT;qMTI
f:.CLIENT;yDZCDf:.CLIENT">
      <style data-impl="1705522452287"></style>
      <div class="L3eUgb" data-hveid="1"> (flex)
        <div class="o3j99 n1xJcf Ne6n5d" role="navigation"></div>
        <div class="o3j99 LLD4me yr19Zb LS8OJ"> (flex)
          <style data-impl="1705522452287"></style>
          <div class="k1zIA rSk4se">
            <style data-impl="1705522452287"></style>
             == $0
          </div>
          <div class="o3j99 ikrT4e om7nvf"></div>
          <div class="o3j99 garstb"></div>
          <div jscontroller="B2qIPe" jsname="cgsKlb" jsaction="rcuQ6b:
npT2md"></div>
          <div class="o3j99 c93Gbe" role="contentinfo"></div>
          <div class="FvgjC"></div>
          <textarea class="csi" name="csi" style="display:none">
          </textarea>
          <script nonce=</script>
          <div class="gb_id" ng-non-bindable=Google Apps</div>
          <div class="gb_q" ng-non-bindable=</div>
          <script src="/js/_/js/k=xjs.hd.en.KT-c626hz_g.O/ckexjs.hd.88V
1U9eh-5o.L1.W/om=IA-V0x.ms4mZb.mu.pFsdhd.pHxghd.q8xT1f.39S54.
g0XF1.gb.wiz.sf.sonic.spch7xjs=s1" nonce async gapi_processed=
"true"></script>
          <script src="/js/_/js/k=xjs.hd.en.KT-c626hz_g.O/ckexjs.hd.88V
1U9eh-5o.L1.W/om=IA-V0x.ms4mZb.mu.pFsdhd.pHxghd.q8xT1f.39S54.
g0XF1.gb.wiz.sf.sonic.spch7xjs=s1" nonce async gapi_processed=
"true"></script>
        </div>
      </div>
    </body>
  </html>
```

Styles Computed Layout Event Listeners

Filter

element.style {

height: 92px;

width: 272px;

img {

height: 92px;

width: 272px;

img {

height: 92px;

width: 272px;

body, input, button {

font-size: 14px;

font-family: arial, sans-serif;

color: #444;



Aqui você terá acesso aos elementos da tela.

Você pode editar o HTML e o CSS diretamente na página e ver as alterações em tempo real.

Editar Ver Histórico Favoritos Perfis Guia Janela Ajuda

gle.com.br

Gmail Imagens

Google

Pesquisa Google Estou com sorte

DevTools is now available in Portuguese! **Always use Chrome's language** Switch DevTools to Portuguese Don't show again

Elements Console Sources **Network** Performance Memory Application

Filter ☐ Preserve log ☐ Disable cache No throttling

All Doc JS Fetch/XHR CSS Font Img Media Manifest WS Wasm Other ☐ Blocked response cookies

☐ Blocked requests ☐ 3rd-party requests

Name	Status	Type	Initiator	Size	Time	Waterfall
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	191 ms	
log?format=json&hasfast=true&au...	200	xhr	rs=AA2YrTv4En...	154 B	345 ms	
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	216 ms	
log?format=json&hasfast=true&au...	200	xhr	rs=AA2YrTv4En...	154 B	368 ms	
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	187 ms	
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	97 ms	
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	111 ms	
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	202 ms	
gen_204?atyp=i&ei=FDWoZYC0E...	204	ping	m=cdoos.hsm.js...	19 B	190 ms	
log?format=json&hasfast=true&au...	200	xhr	rs=AA2YrTv4En...	154 B	359 ms	



Navegue até a aba **Network**.

Aqui você pode ver as requisições HTTP que foram feitas pela página.

- Para limpar as requisições anteriores, vá até **Clear network log**.

DevTools is now available in Portuguese!

Elements Console Source

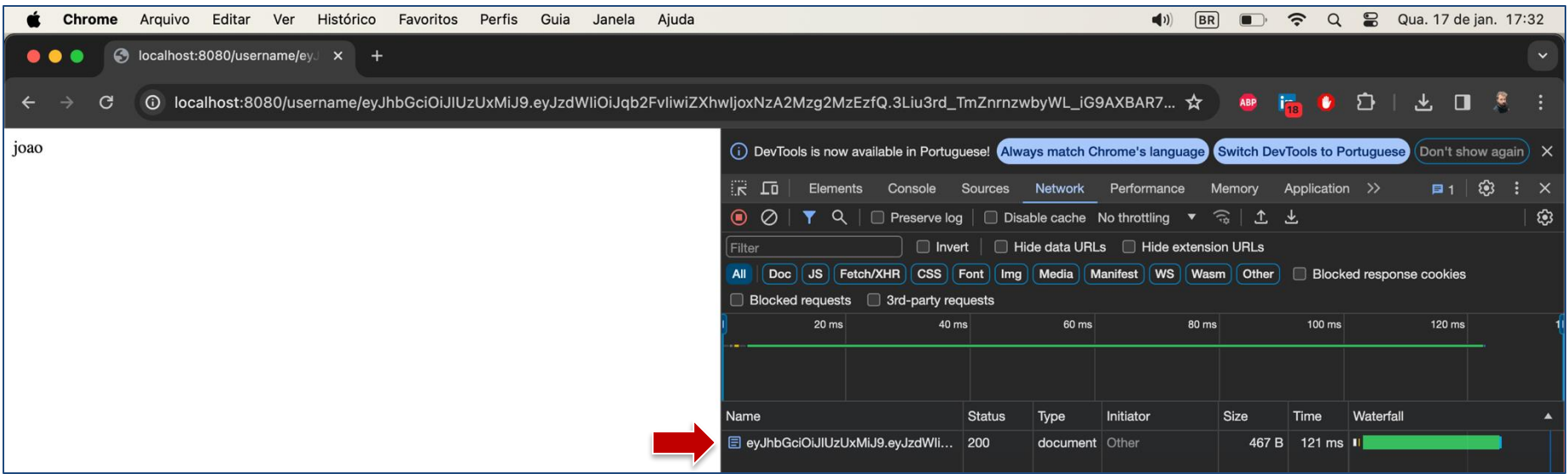
☐ Preserve log ☐

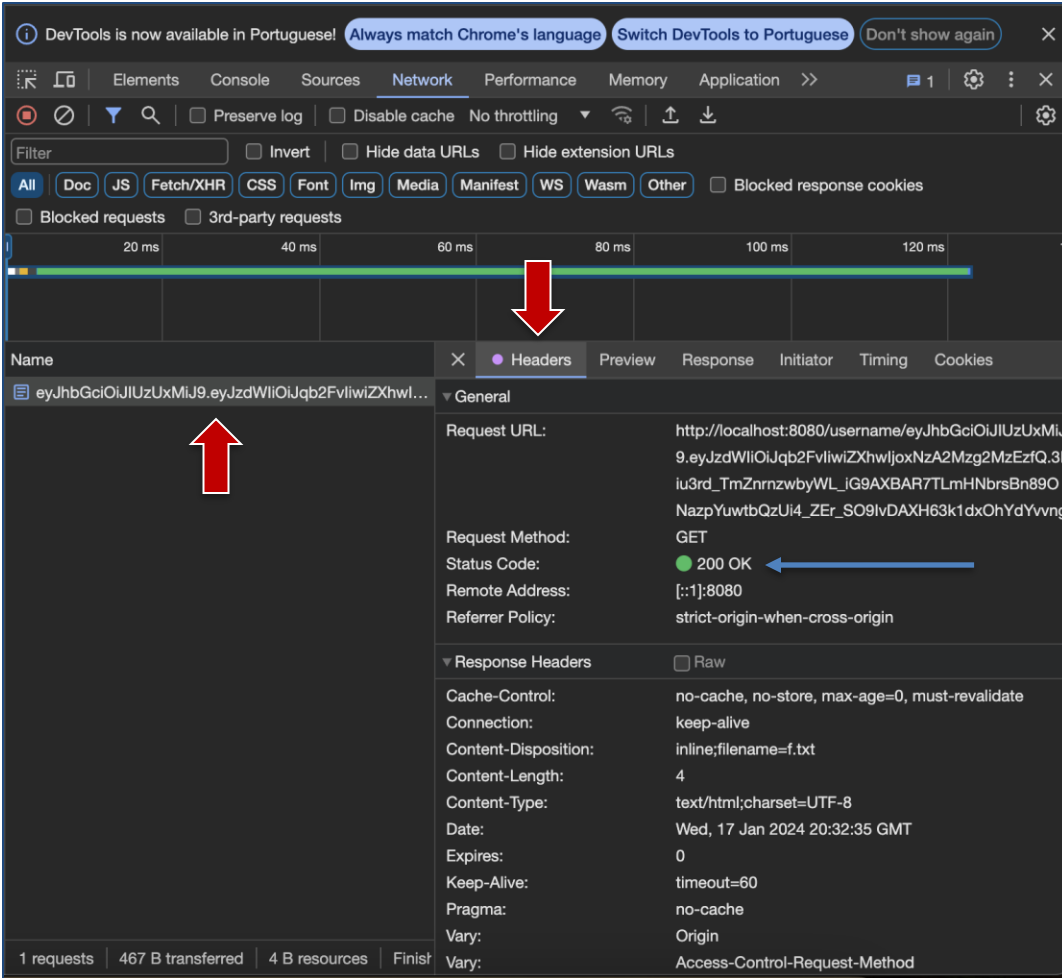
Filter

All Doc JS Fetch/XHR CSS Font



- Agora faça uma requisição, por exemplo, para:
 - <http://localhost:8080/username/{token}>
 - Substitua {token} por um token válido gerado em /login.
- Clique sobre a requisição e navegue em seu Cabeçalho (Headers).

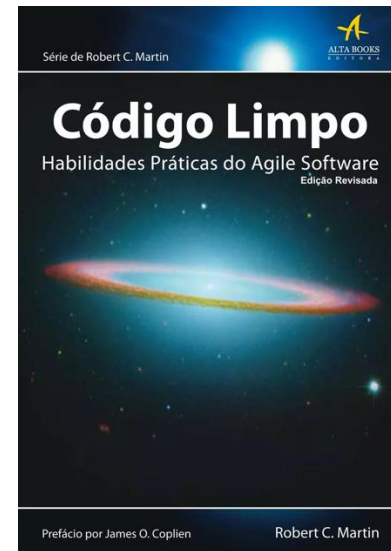
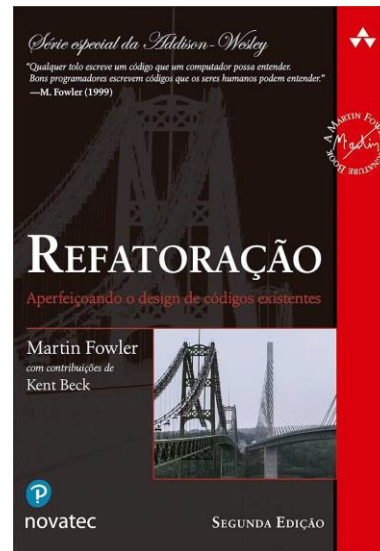
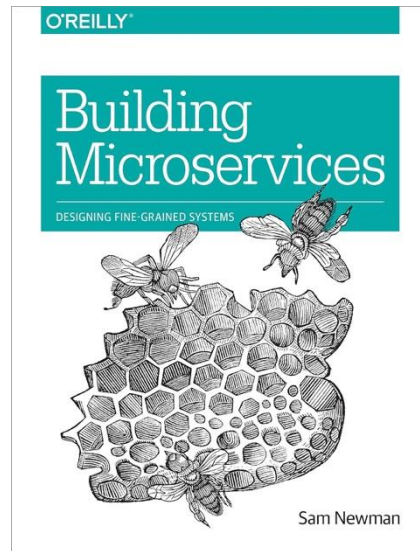




O **DevTools** fornece uma interface rica para inspecionar, depurar e modificar o código HTML, CSS e JavaScript de uma página.

Habitue-se a utilizá-lo.

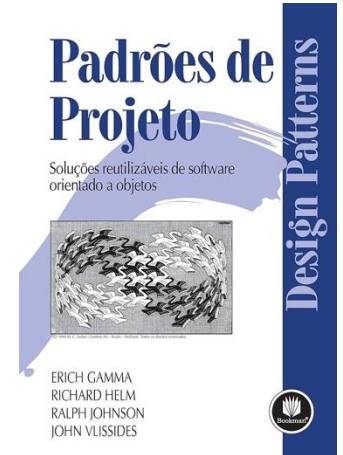
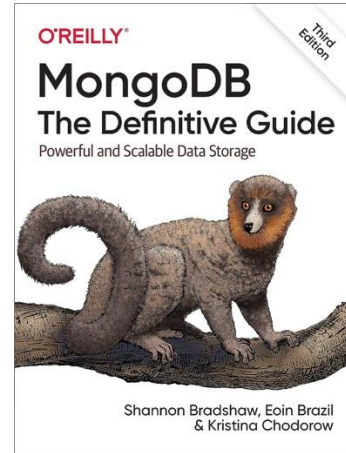
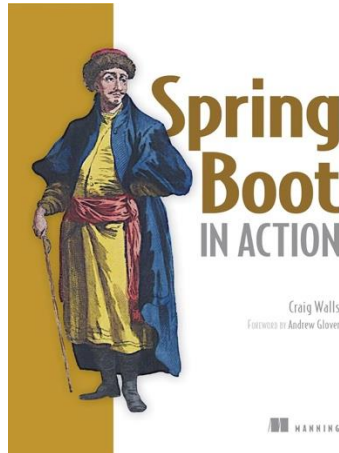
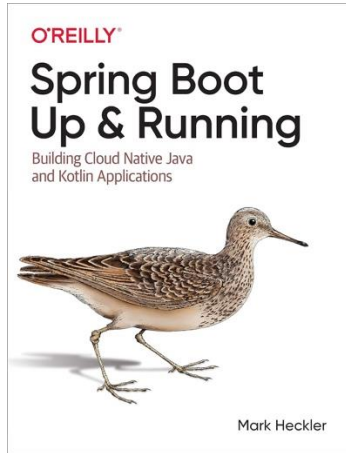
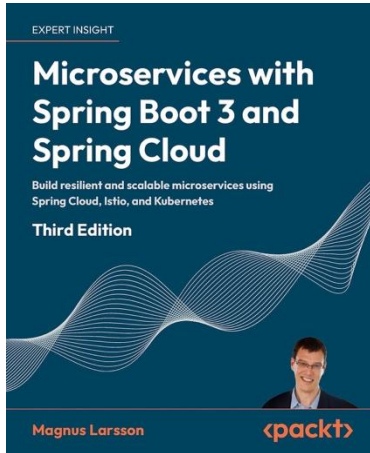
Referências básicas



Referências complementares



Outras referências



Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com



[GitHub](#)



[LinkedIn](#)



[Lattes](#)

...